

INT104 Artificial Intelligence

人工智能

内容基于**LearningMall**的PPT。仅供学习和交流使用。任何机构或个人不得将其用于商业用途或未经授权的传播。如需引用或分享，请注明出处并保留此声明。



GITHUB

REPOSITORY



VISIT NOTES AUTHOR'S PAGE

CLICK ME

使用前声明

INT104 24-25 学年课程设置**有争议**。

仅仅作为课件翻译和总结，对于读者难以理解的公式等，应当适当跳过。

注意这不是一篇面向0基础ML的笔记（因为原PPT内容过于复杂），仅供翻阅

此外LSC表示他的很多东西都是AI-generated的，所以他的课件水分挺多的

■ [Week 1 + Week 2]

第一周是介绍，第二周讲了一些python，可以找到同目录下的 `python.pdf` 查看。

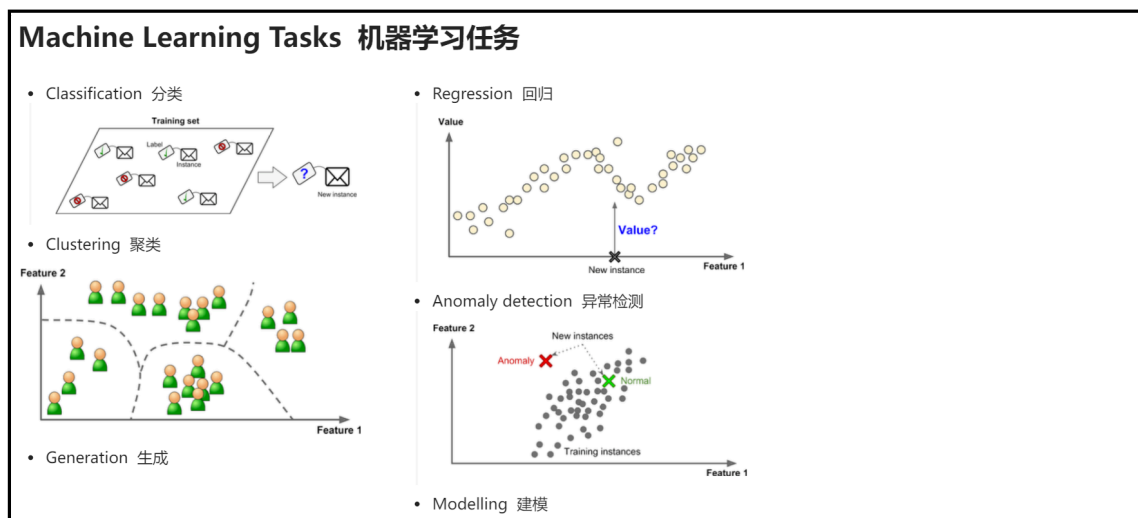
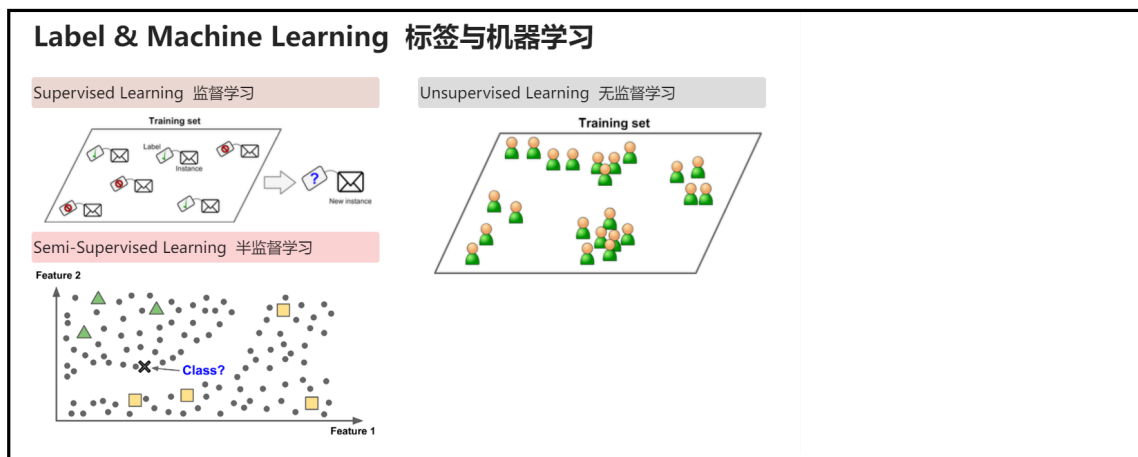
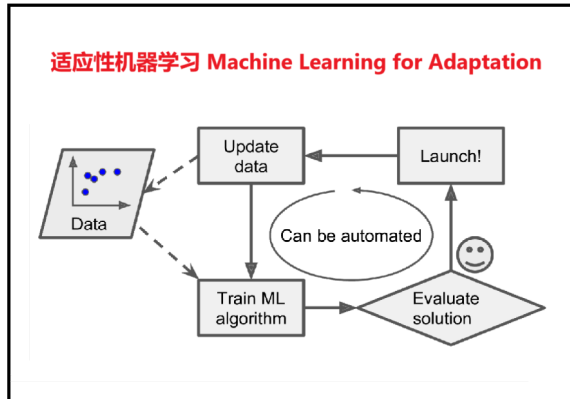
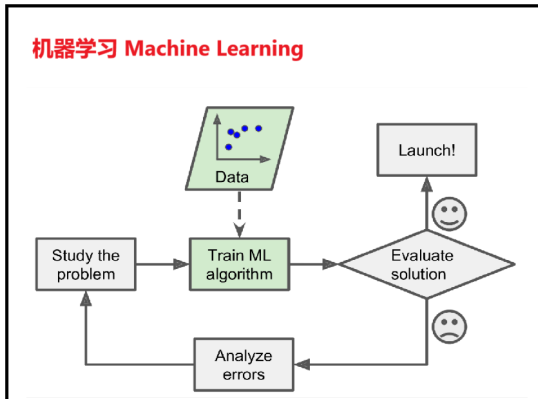
Week	Lecture
Week 1	Overview
Week 2	Python
Week 3	Data Collection & Data Distribution
Week 4	Data Features & PCA
Week 5	Unsupervised Learning I: GMM & DBSCAN
Week 6	Unsupervised Learning II: k-Means & Hierarchical Clustering
Week 7	Supervised Learning I: SVM & Naïve Bayes
Week 8	Supervised Learning II: Decision Tree & kNN
Week 9	Fine-tune & System Evaluation
Week 10	Applications of AI: NLP, CV, Audio etc.
Week 11	
Week 12	AI ethics and future AI development
Week 13	Experiment 3 Show Case & Revision

[0] 课程信息

Teacher	Mail	Office Hour
Dr. Shengchen LI (Module Leader)	Shengchen.Li@xjtlu.edu.cn	4-5 pm Thursdays 10-11 am Fridays
Prof. Dechang XU	Dechang.Xu@xjtlu.edu.cn	9 am Mondays

[1] 简单介绍

关于机器学习...

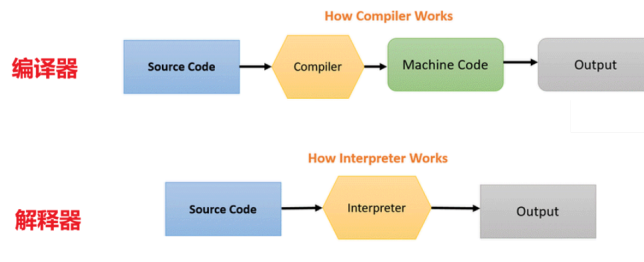


人工智能系统面临的挑战

- Insufficient data 数据不足
- Nonrepresentative Training Data 非代表性训练数据
- Poor data quality 数据质量差
- Irrelevant Features 无关功能
- Overfitting 过拟合

- Underfitting 欠拟合
- Data Mismatch (Data Domain) 数据不匹配（数据域）

[2] 关于python



Python 是解释型语言，不同于Java和C这些编译型语言。

Python需要自己学！本课程不会一步步地教授！课程提供了一些网址：

1. [Learn Python - Free Interactive Python Tutorial](#)
2. [NumPy](#)
3. [SciPy](#)
4. [scikit-learn: machine learning in Python — scikit-learn 1.6.1 documentation](#)

在Week1的Lecture中包含了总共6小时的 python 教程（BigBlueButton），请查看同目录下的 `python.pdf`。

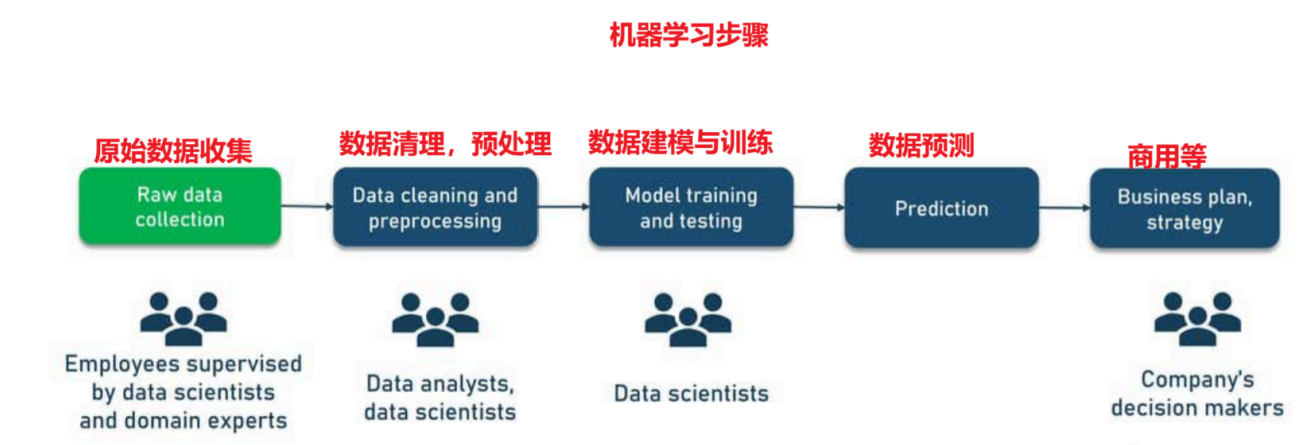
Python可以使命令行、IDE、DeepNote和Jupyter（基于浏览器的IDE）来运行...

■ [Week 3]

数据收集与数据分布 Data Collection & Data Distribution

这节课主要介绍了一些数学概念

[0] 数据训练过程是怎么样的呢？



一些常见的数据结构

	结构化数据（Structured data）	非结构化数据（Unstructured data）	半结构化数据（Semi-structured data）
格式（Formats）	表格、行、列	文本、图像、音频、视频	XML、JSON、CSV
数据模型（Data model）	关系型（Relational）	无	层次型/图型（Hierarchical/Graph）
常见存储方式（Common storages）	关系型数据库（Relational databases）、传统数据仓库（traditional data warehouses）	文件系统（File systems）、数据湖（data lakes）、云数据仓库（cloud data warehouses）	NoSQL数据库（NoSQL databases）
数据性质（Data nature）	定义明确，模式固定（Well-defined, fixed schema）	不可预测，无模式（Unpredictable, no schema）	模式松散（Loose schema）
分析方法（Analysis methods）	SQL查询、数据挖掘（SQL queries, data mining）	自然语言处理（NLP）、图像识别、视频分析、文本分析、音频分析等	查询语言、数据挖掘（Query languages, data mining）
工具和技术（Tools and technologies）	微软SQL Server（Microsoft SQL Server）、Oracle、MySQL	亚马逊S3（Amazon S3）、Hadoop、Spark	MongoDB、Cassandra、Couchbase

	数据仓库（Data warehouses）	数据湖（Data lakes）	数据集市（Data marts）
用途	满足整个组织的数据分析和报告需求	满足不同类型和难度的报告需求、进行预测分析	满足特定运营部门或主题的报告需求
存储的数据（通常）	大量结构化数据；已处理	大量结构化和非结构化数据；原始	有限的结构化数据；已处理
数据源	一系列外部和内部数据源，涵盖不同业务领域	任何外部或内部数据源	与一个业务领域相关的少数数据源

	数据仓库（Data warehouses）	数据湖（Data lakes）	数据集市（Data marts）
大小	大于100GB	大于100GB	小于100GB
创建难度	难以设置	难以设置	容易设置

LLM Data Guide & 6 Methods of Collection in 2025

方法	成本效益	速度	定制化程度	隐私级别
众包	中等	高	高	中等
托管数据收集	低	中等	高	高
自动化数据收集	高	高	中等	低
自动化数据生成（合成数据）	高	高	高	中等
授权数据集	中等	高	低	低
机构合作	中等	中等	高	中等

[1] 数据类型

数值型

Numeric

以数字刻度表示的数据。

- **连续型（Continuous）**：可以在一个区间内取任意值的数据。
（同义词：区间型（interval）、浮点型（float）、数值型（numeric））
- **离散型（Discrete）**：只能取整数值的数据，比如计数。
（同义词：整型（integer）、计数型（count））

分类型

Categorical

只能取一组特定值，这些值代表一系列可能类别的数据。

- （同义词：枚举型（enums）、列举型（enumerated）、因子型（factors）、标称型（nominal））
- **二元型（Binary）**：分类型数据的一种特殊情况，只有两个类别的值，例如 0/1、真 / 假。
（同义词：二项型（dichotomous）、逻辑型（logical）、指示型（indicator）、布尔型（boolean））
- **有序型（Ordinal）**：具有明确顺序的分类型数据。
（同义词：有序因子型（ordered factor））

[2] 矩形数据

(Rectangular Data)

数据框 Data frame

矩形数据（类似电子表格）是统计和机器学习模型的基本数据结构。

特征 Feature

表格中的一列通常被称为一个特征。

同义词：属性（attribute）、输入（input）、预测变量（predictor）、变量（variable）

结果 Outcome

许多数据科学项目都涉及预测一个结果，通常是“是/否”类型的结果。在实验或研究中，特征有时被用来预测结果。

同义词：因变量（dependent variable）、响应（response）、目标（target）、输出（output）

记录 Records

表格中的一行通常被称为一条记录。

同义词：案例（case）、示例（example）、实例（instance）、观测值（observation）、模式（pattern）、样本（sample）

[3] 位置估计量

Estimates of Location

均值 Mean

所有数值的总和除以数值的个数。

同义词：平均数（average）

加权均值 Weighted mean

所有数值乘以相应权重后的总和，再除以权重的总和。

同义词：加权平均数（weighted average）

中位数 Median

将数据分成数量相等的两部分，该数值恰好在中间位置。

同义词：第50百分位数（50th percentile）

百分位数 Percentile

使得数据中 $P\%$ 的数据小于该数值。

同义词：分位数（quantile）

加权中位数 Weighted median

在已排序的数据中，使得权重总和的一半分别位于该数值之上和之下的数值。

假设我们有一组数据及其对应的权重：

数据值 x	权重 w
3	2
5	3
7	1
9	4

首先，将数据按照数据值从小到大排序：

数据值 x	权重 w	累计权重 C_w
3	2	2
5	3	2+3=5
7	1	5+1=6
9	4	6+4=10

权重总和为 $2 + 3 + 1 + 4 = 10$ ，一半的权重总和是 $10/2 = 5$ 。

从累计权重可以看出，当取到数据值 $x = 5$ 时，其累计权重为 5，满足“**权重总和 (2 + 3 + 1 + 4 = 10) 的一半分别位于该数值之上和之下**”（小于 5 的数据对应的权重累计为 2，大于 5 的数据对应的权重累计为 1+4=5）。所以这组数据的加权中位数就是 5。

截尾均值 Trimmed mean

剔除一定数量的极端值后，所有数值的平均值。

同义词：切尾均值（truncated mean）

稳健的 Robust

对极端值不敏感。

同义词：抗干扰的（resistant）

离群值 Outlier

与大多数数据值差异很大的数据值。

同义词：极端值（extreme value）

[4] 变异性度量

Variability Metrics

偏差 Deviations

观测值与位置估计量之间的差值。

同义词：误差（errors）、残差（residuals）

方差 Variance

各数据值与均值之差的平方和，除以 $n - 1$ ，其中 n 是数据值的数量。

同义词：均方误差 mean-squared-error

标准差 Standard deviation

方差的平方根。

平均绝对偏差 Mean absolute deviation

各数据值与均值之差的绝对值的平均值。

同义词： $L1$ 范数（ $l1$ -norm）、曼哈顿范数（Manhattan norm）

中位数绝对偏差 Median absolute deviation from the median

各数据值与中位数之差的绝对值的中位数。

极差 Range

数据集中最大值与最小值之间的差值。

顺序统计量 Order statistics

基于将数据值从小到大排序后得到的度量。

同义词：秩（ranks）

百分位数 Percentile

这样一个数值，使得数据中 $P\%$ 的数值小于等于该值， $(100 - P)\%$ 的数值大于等于该值。

同义词：分位数 (quantile)

四分位距 (Interquartile range)

第75百分位数与第25百分位数之间的差值。

同义词：IQR (Interquartile Range)

自由度，以及分母该用 n 还是 $n - 1$?

在统计学书籍中，总会讨论为什么方差公式的分母是 $n - 1$ ，而不是 n ，这就引出了自由度的概念。不过这种区别其实并不重要，因为通常 n 足够大，除以 n 还是 $n - 1$ 不会产生太大差异。但如果你感兴趣，下面来解释一下。这基于你想通过样本对总体进行估计的前提。

为什么使用 $n - 1$ 而不是 n ?

1. 无偏估计：

如果在方差公式中使用直观的分母 n ，你会低估总体方差和标准差的真实值，这被称为有偏估计。然而，如果用 $n - 1$ 代替 n 作为分母，方差就会成为无偏估计。

2. 自由度的概念：

要完整解释为什么使用 n 会导致有偏估计，就涉及到自由度的概念，自由度考虑了在计算估计值时的约束条件数量。在这种情况下，有 $n - 1$ 个自由度，因为存在一个约束条件：标准差的计算依赖于样本均值。

总结

对于大多数问题，数据科学家无需担心自由度的问题。当样本量 n 较大时，除以 n 还是 $n - 1$ 的差异可以忽略不计。

[5] 探索数据分布

箱线图 Boxplot

由图基 (Tukey) 提出的一种能快速可视化数据分布的图形。

同义词：盒须图 (box and whiskers plot)

频率表 Frequency table

对落入一组区间 (组距) 内的数值型数据值的计数统计。

直方图 Histogram

一种以组距为横轴，计数（或比例）为纵轴的频率表图形。虽然在视觉上相似，但条形图不应与直方图混淆。

密度图 Density plot

直方图的平滑版本，通常基于核密度估计。

[6] 探索分类数据

众数 Mode

数据集中出现频率最高的类别或数值。

期望值 Expected value

当各个类别可与一个数值相关联时，期望值是基于某一类别出现的概率得出的平均值。

条形图 Bar charts

将每个类别的频率或比例以条形的形式绘制出来的图表。

饼图 Pie charts

将每个类别的频率或比例以扇形的形式绘制在一个圆形中呈现的图表。

[7] 相关性

相关系数 Correlation coefficient

一种用于衡量数值型变量之间相互关联程度的度量（取值范围从 -1 到 +1）。

相关矩阵 Correlation matrix

一种表格，其中行和列均表示变量，单元格中的值是变量之间的相关系数。

$$r = \frac{n(\sum xy) - (\sum x)(\sum y)}{\sqrt{[n \sum x^2 - (\sum x)^2][n \sum y^2 - (\sum y)^2]}}$$



Karl Pearson

散点图 Scatterplot

一种图形，其中 x 轴表示一个变量的值，y 轴表示另一个变量的值。

探索两个或多个变量

列联表 Contingency table

对两个或多个分类变量之间的计数统计列表。

六边形分箱图 Hexagonal binning

一种展示两个数值型变量的图形，记录数据被划分到六边形区域中。

等高线图 Contour plot

一种像地形图一样，展示两个数值型变量密度的图形。

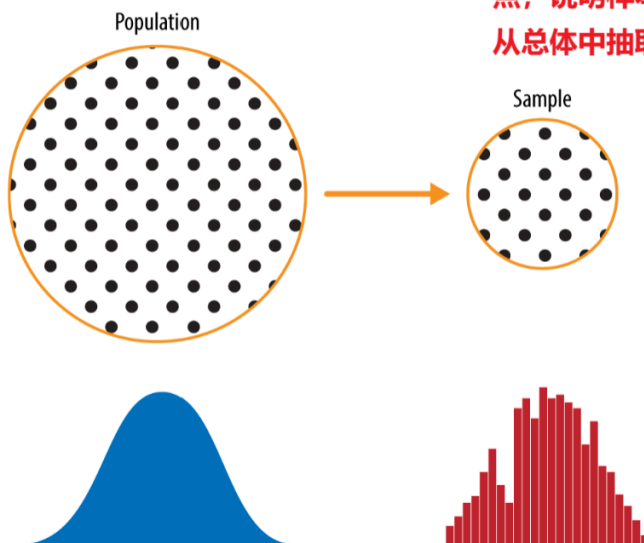
小提琴图 Violin plot

与箱线图类似，但会展示密度估计。

[8] 总体和样本

左边是总体，包含了所有的黑色小点，意味着总体是集合...包含了研究对象这些小个体

右边代表样本，是从总体中抽取的一部分黑色小点，说明样本是子集，从总体中选出的。箭头表示从总体中抽取样本的过程。



[9] 随机抽样

样本 Sample

从更大的数据集中选取的一个子集。

总体 Population

更大的数据集，或对数据集的一种概念性表述。

N (n)

总体（样本）的规模大小。

随机抽样 Random sampling

随机地从总体中抽取元素组成样本。

分层抽样 Stratified sampling

将总体划分为不同的层，然后从每一层中随机抽样。

层（Stratum，复数strata）

总体中具有共同特征的同类子群体。

简单随机样本 Simple random sample

在不对总体进行分层的情况下，通过随机抽样得到的样本。

偏差 Bias

系统性误差。

样本偏差 Sample bias

无法准确代表总体特征的样本。

选择偏差 Select Bias

1. 选择偏差 Selection bias

因观测对象选取方式而产生的偏差。

2. 数据窥探 Data snooping

对数据进行大量搜寻，以寻找有趣的信息。

3. 大规模搜索效应 Vast search effect

由于重复进行数据建模，或使用大量预测变量对数据进行建模而导致的偏差或不可重复性。

[10] 抽样分布

Sampling Distribution

样本统计量 Sample statistic

从较大总体中抽取的样本数据所计算得出的一种度量指标。

数据分布 Data distribution

数据集中单个数值的频率分布。

抽样分布 Sampling distribution

在多个样本或重抽样中，样本统计量的频率分布。

中心极限定理 Central limit theorem

随着样本量增加，抽样分布趋于正态分布的一种趋势。

标准误差 Standard error

多个样本中样本统计量的变异性（标准差）（不要与标准差混淆，标准差本身指的是单个数据值的变异性）。

[11] 自助法

Bootstrap

自助样本 Bootstrap sample

从观测数据集中有放回抽样得到的样本。

重抽样 Resampling

从观测数据中反复抽取样本的过程，包括自助法和排列（打乱）法。

[12] 置信区间

Confidence Intervals

置信水平 Confidence level

以相同方式从同一总体中构造的置信区间中，预期包含感兴趣统计量的比例。

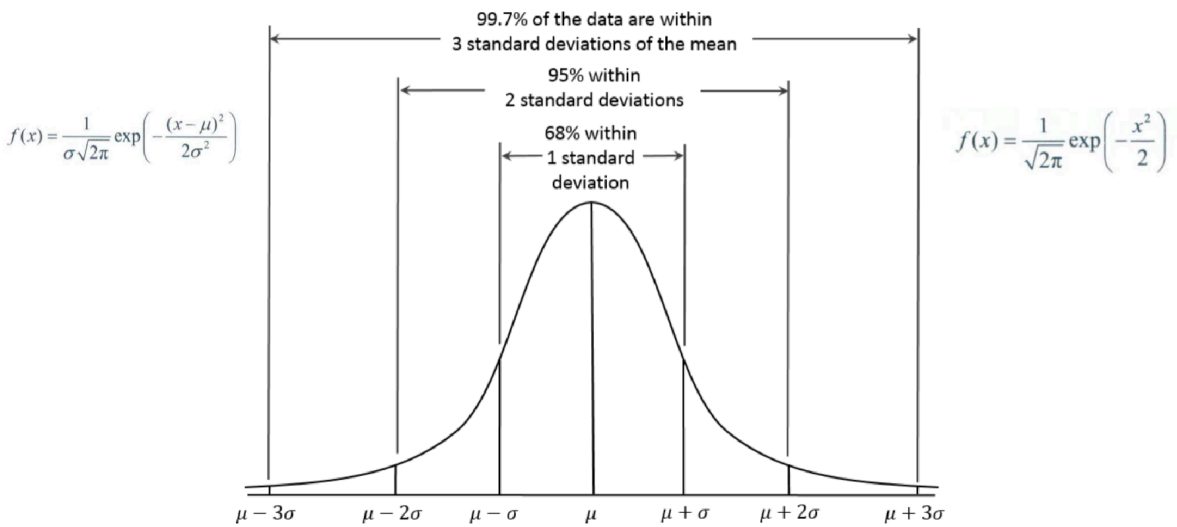
区间端点 Interval endpoints

置信区间的上限和下限。

[13] 正态分布

Normal Distribution 正态曲线 Normal Curve

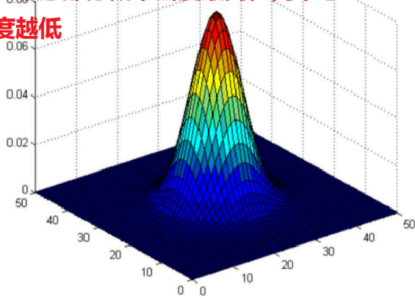
μ 代表均值，是正态分布的中心位置；
 σ 代表标准差，衡量数据的离散程度。



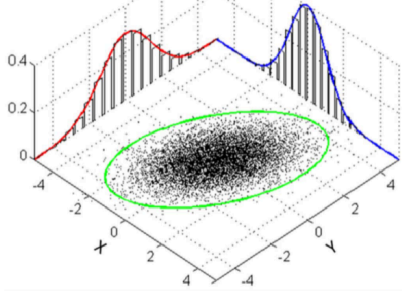
正态曲线 (Normal curve)

左侧图像：是一个二维正态分布的概率密度函数的立体图，体现了二维正态分布在平面上不同点的概率密度值。

在二维正态分布中，数据点的概率密度呈现出类似山峰的形状，中心部分概率密度最高，离中心越远，概率密度越低
注意是二维的



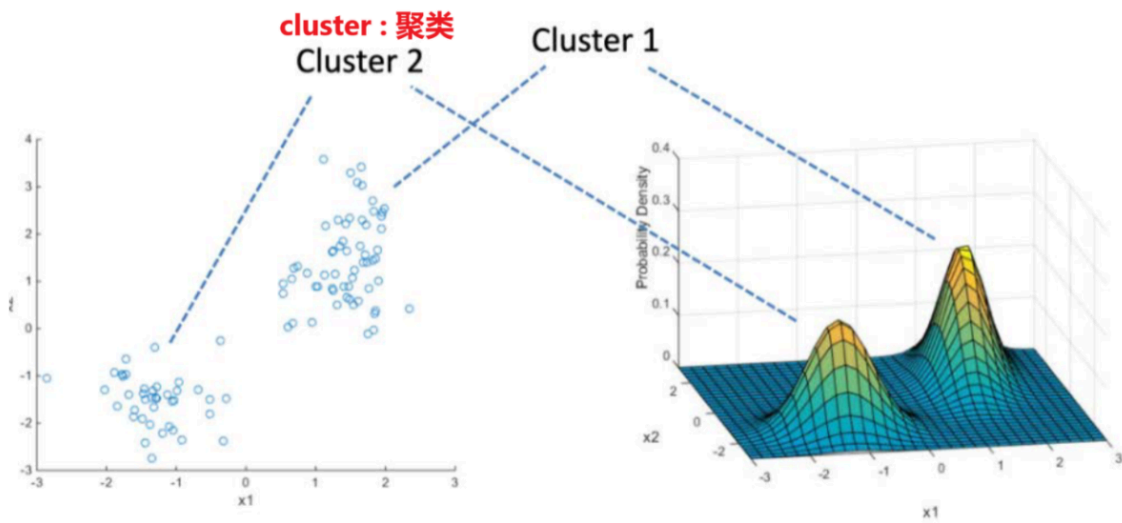
右侧图像：展示了二维正态分布的数据点分布情况，绿色椭圆区域内分布着大量的数据点。两侧红色和蓝色曲线分别是变量 X 和 Y 的边缘分布，体现了每个变量单独的概率分布情况。



* 边缘分布是指在多维概率分布中，不考虑其他变量，仅关注某一个变量的概率分布情况。

$(X, Y) \sim N(\mu_1, \mu_2, \sigma_1^2, \sigma_2^2, \rho)$

$$f(x, y) = \frac{1}{2\pi\sigma_1\sigma_2\sqrt{1-\rho^2}} \cdot \exp\left(\frac{-1}{2(1-\rho^2)}\left[\frac{(x-\mu_1)^2}{\sigma_1^2} - 2\rho\frac{(x-\mu_1)(y-\mu_2)}{\sigma_1\sigma_2} + \frac{(y-\mu_2)^2}{\sigma_2^2}\right]\right)$$



它是无监督学习的一种重要方法，目的是将数据集中的样本按照相似性划分为不同的组（即簇），使得同一簇内的样本相似度较高，而不同簇间的样本相似度较低。比如在高斯混合模型（GMM）和 DBSCAN 算法中，都是通过特定的方式对数据进行处理，从而得到不同的 cluster，帮助发现数据中的内在结构和模式。

误差 Error

数据点与预测值或平均值之间的差值。

标准化 Standardize

减去均值，然后除以标准差。

z分数 z-score

对单个数据点进行标准化后的结果。

标准正态分布 Standard normal

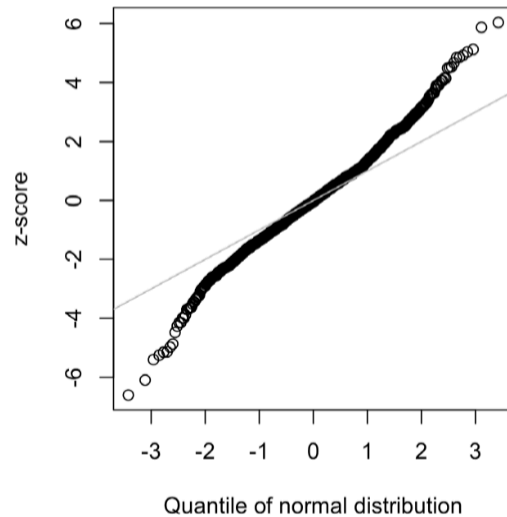
均值为0、标准差为1的正态分布。

QQ图 QQ-Plot

一种用于可视化样本分布与指定分布（如正态分布）接近程度的图形。

[14] 长尾分布

Long-Tailed Distributions



**奈飞 (Netflix, 股票代码 NFLX) 收益率的 QQ-Plot
这个属于长尾分布**

尾部 Tail

频率分布中细长的部分，在该部分相对极端的值出现的频率较低。

偏度 Skew

指分布的一侧尾部比另一侧更长的情况。

[15] 学生 t 分布

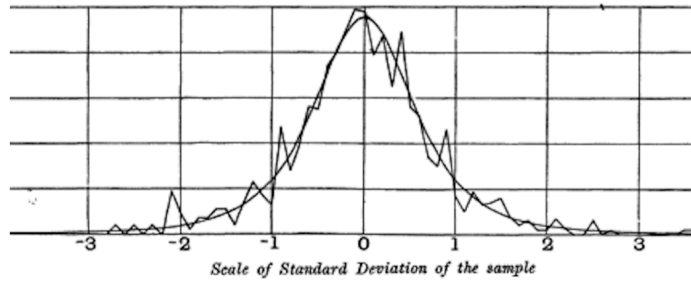
Student's t distribution

n

样本量

自由度

一个能让 t 分布根据不同样本量、统计量和组数进行调整的参数。



戈塞特 (Gosset) 的重抽样实验

实验结果及拟合的t曲线 (来自他1908年发表于《生物统计学》期刊的论文)

[16] 二项分布

Binomial Distribution

试验 Trial

具有离散结果的一个事件 (例如, 抛硬币)。

成功 Success

试验中所关注的结果。

同义词: “1”(相对于“0”而言)

二项的 Binomial

具有两种结果的。

同义词: 是/否, 0/1, 二元的

二项试验 Binomial trial

有两种结果的试验。

同义词: 伯努利试验

二项分布 Binomial distribution

在 X 次试验中成功次数的分布。

同义词: 伯努利分布

[17] 泊松分布及相关分布

Poisson and Related Distributions

兰布达 Lambda

事件发生的速率 (每单位时间或空间)。

泊松分布 Poisson distribution

在抽样的单位时间或空间内事件发生数量的频率分布。

指数分布 Exponential distribution

从一个事件到下一个事件的时间或距离的频率分布。

威布尔分布 Weibull distribution

指数分布的一种广义形式，其中事件发生的速率允许随时间变化。

■ [Week 4]

数据特征与主成分分析 Data Features & Principal Component Analysis

这节课讲了PCA的原理（很复杂），以及如何用Python实现PCA（也很复杂）

一个示例是 [鸢尾花数据集上的主成分分析（PCA）](#)，用到的数据集[在这里](#)。



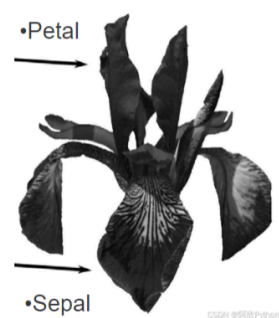
•Iris Setosa



•Iris Versicolor



•Iris Virginica



数据特征

特征 Feature

表格中的一列 (row, tuple) 通常被称为一个特征。

同义词：属性（Attribute）、输入（Input）、预测变量（Predictor）、变量（Variable）

机器学习中的主成分分析

Principal Component Analysis in Machine Learning

主成分

预测变量的线性组合。

载荷 Loadings

将预测变量转换为主成分的权重。

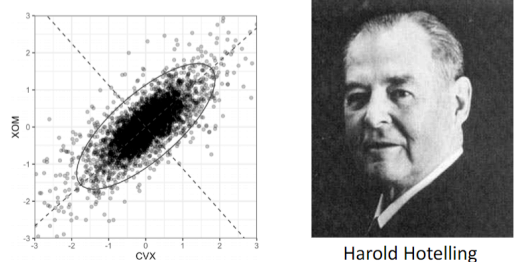
同义词：权重

碎石图

一种展示主成分方差的图，以解释方差或解释方差的比例的形式，呈现各主成分的相对重要性。

主成分分析（PCA）的主要目标是在尽可能保留信息的前提下，**减少数据集中的变量数量**。

机器学习中的主成分分析主要可用于降维和重要特征选择。将相关特征转化为独立特征。



为了进行全面、系统的分析和研究，需要考虑众多指标，这些指标能够从不同方面反映研究对象的特征，但指标过多会增加分析的复杂性。

能否将原始变量精简为几个具有代表性的新变量，来代表原来的指标呢？

示例引入

约翰·理查德·尼古拉斯·斯通 是一位具有开创性的经济学家，他开发了一种**系统的复式记账法**用于国民收入核算。他在 1984 年被授予诺贝尔经济学奖。

他使用了美国 1929 年至 1938 年的数据，**得出了 17 个反映国民收入和支出的变量**，如雇主补贴、消费和生产资料、公共净支出、存货净增加、股息、利息以及对外贸易差额。

经过主成分分析（PCA）后，原本的17个变量被降维为3个新的变量，降维后的数据保留了97.4的原始信息。基于经济学理论，斯通将这三个新的主成分分别命名为：

- 1. **总收入** (F_1)：表征经济中的总收入水平。
- 2. **总收入变化率** (F_2)：表征总收入的变化速度。
- 3. **经济发展或衰退趋势** (F_3)：表征经济是处于增长还是衰退的趋势。

值得注意的是，这三个变量实际上是可以直接测量的。斯通 进一步对这三个主成分与实际可测量的指标进行了相关性分析，具体包括：

- 实际测量的总收入 (I)
- 总收入变化率 (ΔI)
- 时间因素 (t)

	F_1	F_2	F_3	I	ΔI	t
F_1	1					
F_2	0	1				
F_3	0	0	1			
I	0.995	-0.041	0.057	1		
ΔI	-0.056	0.948	-0.124	-0.102	1	
t	-0.369	-0.282	-0.836	-0.414	-0.112	1

总结：主成分分析旨在寻找一种方法，**用较少的新变量替代较多的旧变量**，同时让新变量尽可能多地保留信息。

有 n 个样本 (row)，每个样本共有 p 维空间 (column)。

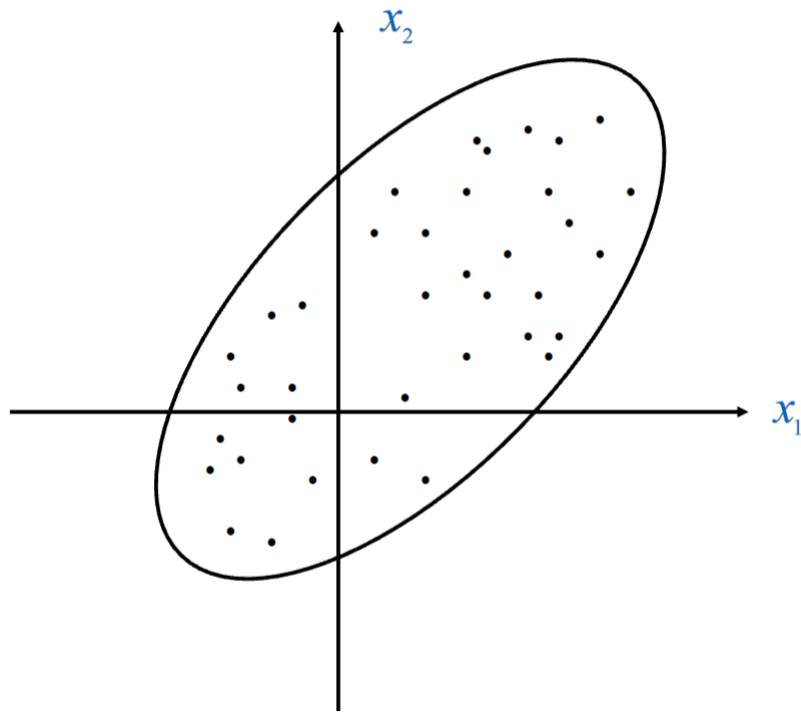
$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

这些变量 (column) 构成一个 $n \times p$ 阶的数据矩阵。

但是这样还是太多了！为了克服这一困难，有必要进行降维处理，即用较少的综合指标替代原始指标，并使这些综合指标尽可能多地反映原始指标所表达的信息，同时，这些综合指标相互独立。

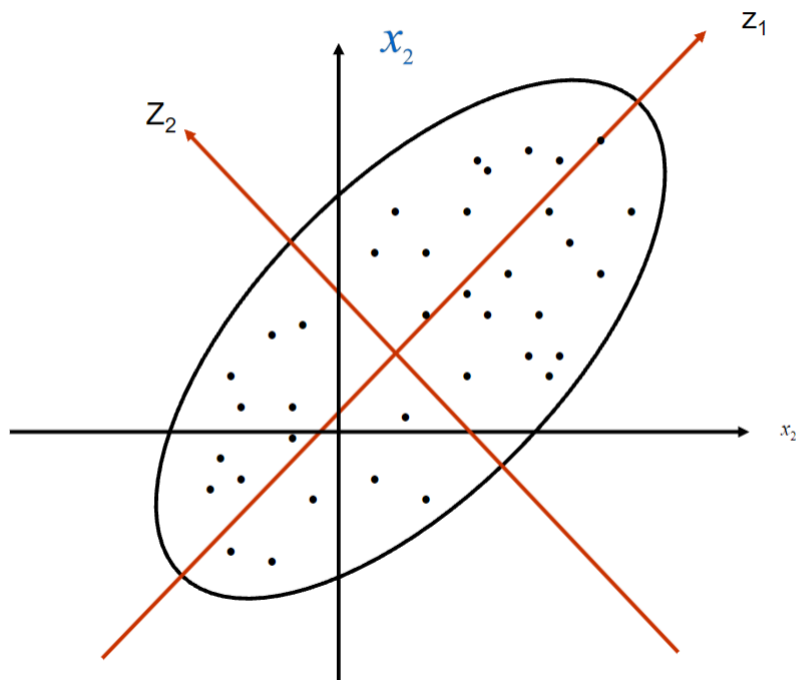
在选择综合指标时，最简单的形式是对原始变量进行**线性组合 Linear Combination**，适当调整组合系数，使新变量相互独立且具有代表性。

为便于理解，我们讨论二维空间中主成分的几何意义。假设有 n 个样本，每个样本有两个观测变量 x_1 和 x_2 。在由变量 x_1 和 x_2 确定的二维平面中， n 个样本点呈椭圆状散布。



可以看出，这 n 个样本点在 x_1 轴或 x_2 轴方向上具有很大的离散度，离散度的大小可以分别用观测变量 x_1 和 x_2 的方差来定量表示。显然，如果只考虑 x_1 或 x_2 中的一个，原始数据所包含的信息将会有显著损失。

如果我们同时平移 x_1 轴和 x_2 轴，并将它们逆时针旋转角度 θ ，就会得到新的坐标轴 z_1 和 z_2 ， z_1 和 z_2 是两个新变量。



$$\begin{cases} z_1 = x_1 \cos \theta + x_2 \sin \theta \\ z_2 = -x_1 \sin \theta + x_2 \cos \theta \end{cases}$$

根据旋转变换公式：

$$\begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = U'x$$

其中 U 是一个**正交矩阵 (orthogonal matrix)** 也就是说

$$U' = U^{-1}, U'U = E$$

z_1 、 z_2 不仅能够集中 x_1 、 x_2 所包含的信息，而且还具有不相关的性质，这使得在研究复杂问题时能够避免因信息重叠而导致的虚假性。

二维平面上各点的方差大多归因于 z_1 轴，而 z_2 轴上的方差则非常小。

z_1 和 z_2 被称为原始变量 x_1 和 x_2 的综合变量。 z 简化了系统的结构并抓住了主要矛盾。

旋转变换的目的是使 n 个样本点在 z_1 轴方向上的离散程度达到最大化，也就是说， z_1 的方差最大，并且变量 z_1 代表了原始数据的大部分信息。在研究某些问题时，即便不考虑变量 z_2 ，也几乎不会丢失什么信息。

z_1 被称为**第一主成分**， z_2 被称为**第二主成分**。

总结

设 $x_1, x_2, \dots, x_p$ 为原始变量指标， $z_1, z_2, \dots, z_m$ ($m \leq p$) 为新变量指标，则

$$\begin{cases} z_1 = l_{11}x_1 + l_{12}x_2 + \cdots + l_{1p}x_p \\ z_2 = l_{21}x_1 + l_{22}x_2 + \cdots + l_{2p}x_p \\ \vdots \\ z_m = l_{m1}x_1 + l_{m2}x_2 + \cdots + l_{mp}x_p \end{cases}$$

$z_1, z_2, \dots, z_m$ 被称为原始变量指标 $x_1, x_2, \dots, x_p$ 的第一、第二、...、第 m 主成分。

z_i 被称为 x_i 的第 i 主成分, x_i 是原始变量指标

因此, 主成分分析的主要任务是确定主成分 z_i ($i = 1, 2, \dots, m$) 中关于系数 l_{ij} 的原始变量 x_j ($j = 1, 2, \dots, p$)。

j 是原始变量 x 的下标, i 是主成分 z 的下标。现在必须确认一个 x 和 z 的系数, 这个系数就是 l_{ij} 。这样就可以关联 z 和 多个 x ...

确定系数 l_{ij} 的原则

① z_i 和 z_j ($i \neq j; i, j = 1, 2, \dots, m$) 相互独立;

② z_1 是 $x_1, x_2, \dots, x_p$ 所有线性组合中方差最大的, z_2 是 $x_1, x_2, \dots, x_p$ 所有线性组合中方差最大且与 z_1 相互独立的;

z_m 是 $x_1, x_2, \dots, x_p$ 所有线性组合中方差最大且与 $z_1, z_2, \dots, z_{m-1}$ 相互独立的。

如何计算主成分?

计算相关系数矩阵

原始变量之间的相关性

相关系数矩阵 R 为:

$$R = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1p} \\ r_{21} & r_{22} & \cdots & r_{2p} \\ \vdots & \vdots & \vdots & \vdots \\ r_{p1} & r_{p2} & \cdots & r_{pp} \end{bmatrix}$$

其中, $i, j = 1, 2, \dots, p$, r_{ij} 是原始变量 x_i 和 x_j 的相关系数, 且 $r_{ij} = r_{ji}$, 其计算公式为:

$$r_{ij} = \frac{\sum_{k=1}^n (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)}{\sqrt{\sum_{k=1}^n (x_{ki} - \bar{x}_i)^2 \sum_{k=1}^n (x_{kj} - \bar{x}_j)^2}}$$

计算特征值和特征向量

① 求解特征方程 $|\lambda E - R| = 0$, 找到特征值, 并按大小顺序排列, 即 $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$ 。

② 分别找到与特征值 λ_i ($i = 1, 2, \dots, p$) 对应的特征向量 e_i , 要求 $\|e_i\| = 1$, 也就是 $\sum_{j=1}^p e_{ij}^2 = 1$, 其中 e_{ij} 表示向量 e_i 的第 j 个分量。

计算主成分贡献率和累计贡献率

Contribution rate

贡献率： $\frac{\lambda_i}{\sum_{k=1}^p \lambda_k} \quad (i = 1, 2, \dots, p)$

累计贡献率： $\frac{\sum_{k=1}^i \lambda_k}{\sum_{k=1}^p \lambda_k} \quad (i = 1, 2, \dots, p)$

一般来说，选取累计贡献率达到80%－90%所对应的第1、第2、.....第*m*个主成分值，即选取 $\lambda_1, \lambda_2, \dots, \lambda_m$ 。

- 计算主成分载荷
 $p(z_i, x_j) = \sqrt{\lambda_i} e_{ij} \quad (i, j = 1, 2, \dots, p)$
- 每个主成分的得分：

$$Z = \begin{bmatrix} z_{11} & z_{12} & \cdots & z_{1m} \\ z_{21} & z_{22} & \cdots & z_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ z_{n1} & z_{n2} & \cdots & z_{nm} \end{bmatrix}$$

计算主成分得分

将标准化数据 $(z_1, z_2, \dots, z_p)'$ 代入主成分表达式，得到第一主成分的得分，进而得到每个样本单元在各主成分上的得分。

根据主成分得分的大小，可以分析每个样本单元在各主成分上的表现。

综合得分排名

每个主成分代表分析对象在某些方面的表现。所选取的 *m* 个主成分代表了分析对象的大部分信息，对主成分的综合分析等同于对所有分析对象的综合分析。以 *m* 个主成分的方差贡献率（特征根）为权重，对 *m* 个主成分进行加权平均，加权平均公式如下：

$$F = \frac{\alpha_1 Y_1 + \alpha_2 Y_2 + \cdots + \alpha_m Y_m}{\alpha_1 + \alpha_2 + \cdots + \alpha_m}$$

或者

$$F = \frac{\lambda_1 Y_1 + \lambda_2 Y_2 + \cdots + \lambda_m Y_m}{\lambda_1 + \lambda_2 + \cdots + \lambda_m}$$

得到主成分的综合得分 *F*，该得分保留了大部分信息，其数值大小能够反映分析对象的综合表现。根据主成分得分的高低来评判每个对象的表现情况，从而达到排名的目的。

示例

有三个指标 X_1 、 X_2 、 X_3 ，得到样本相关系数矩阵 (coefficient matrix) *R*为：

$$R = \begin{pmatrix} 1 & 0.632 & -0.867 \\ 0.632 & 1 & -0.331 \\ -0.867 & -0.331 & 1 \end{pmatrix}$$

对应的特征方程为

$$\begin{vmatrix} \lambda - 1 & -0.632 & 0.867 \\ -0.632 & \lambda - 1 & 0.331 \\ 0.867 & 0.331 & \lambda - 1 \end{vmatrix} = 0$$

展开行列式 (determinant) 并计算后得到关于 λ 的特征方程：

$$\lambda^3 - 3\lambda^2 + 1.74\lambda - 0.103 = 0$$

并且可以得到三个特征根：

$$\lambda_1 = 2.245, \lambda_2 = 0.688, \lambda_3 = 0.067$$

第一个特征值 $\lambda_1 = 2.245$ 对应的特征向量 α_1 应满足方程 $(2.245E - R)\alpha_1 = 0$ $\alpha_1 = \begin{pmatrix} \alpha_{11} \\ \alpha_{21} \\ \alpha_{31} \end{pmatrix}$ 且

$$\alpha_{11}^2 + \alpha_{21}^2 + \alpha_{31}^2 = 1$$

因此

$$\begin{pmatrix} 2.245 - 1 & -0.632 & 0.867 \\ -0.632 & 2.245 - 1 & 0.331 \\ 0.867 & 0.331 & 2.245 - 1 \end{pmatrix} \begin{pmatrix} \alpha_{11} \\ \alpha_{21} \\ \alpha_{31} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

也就是说

$$\begin{cases} 1.245\alpha_{11} - 0.632\alpha_{21} + 0.867\alpha_{31} = 0 \\ -0.632\alpha_{11} + 1.245\alpha_{21} + 0.331\alpha_{31} = 0 \\ 0.867\alpha_{11} + 0.331\alpha_{21} + 1.245\alpha_{31} = 0 \end{cases}$$

得到：

$$\alpha_{11} = 0.6520, \alpha_{21} = 0.4857, \alpha_{31} = 0.5822$$

那么，第一主成分是 $F_1 = 0.652X_1 + 0.4857X_2 + 0.5822X_3$ 通过同样的方法，分别针对 λ_2 、 λ_3 可以得到对应的特征向量 $(\alpha_{12}, \alpha_{22}, \alpha_{32})$ 和 $(\alpha_{13}, \alpha_{23}, \alpha_{33})$ 。

由此，得到第二主成分 F_2 和第三主成分 F_3 。

因此

- 在创建第一个主成分时，主成分分析（PCA）得出预测变量的线性组合，该组合能最大化总方差解释率。
- 这个线性组合随后成为第一个“新”的预测变量 Z_1 。
- 主成分分析重复此过程，使用相同的变量但赋予不同的权重，以创建第二个新的预测变量 Z_2 。权重的设定要使得 Z_1 和 Z_2 不相关。
- 这个过程持续进行，直到得到与原始变量 X_i 数量相同的新变量或成分 Z_i 。
- 选择保留足够数量的成分，以解释大部分方差。
- 到目前为止，结果是每个成分的一组权重。最后一步是通过将这些权重应用于原始值，将原始数据转换为新的主成分得分。这些新得分随后可以用作精简后的预测变量集。

为什么在机器学习中需要主成分分析 PCA?

处理高维数据会导致过拟合问题，我们将使用降维的方法来解决这些问题。

PCA能够增强可解释性并最小化信息损失，有助于定位重要特征，还有助于发现不同序列的线性组合。

在机器学习中何时应该使用主成分分析？

- 当我们需要了解特征之间是否相互独立时；
- 当我们需要从大量特征中提取较少的特征时。

机器学习中主成分分析 PCA 的Python代码

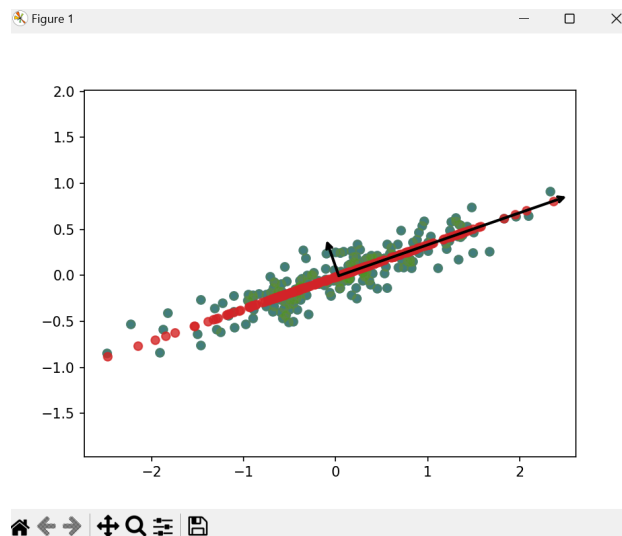
完整代码

```
1  #导入了numpy用于数值计算
2  import numpy as np
3  # matplotlib.pyplot用于数据可视化
4  import matplotlib.pyplot as plt
5
6  # 设置随机数种子，保证结果可复现
7  rng = np.random.RandomState(1)
8  # 生成一个二维的数据集x，它是两个随机矩阵的乘积转置，包含 200 个样本，每个样本有 2 个特征。
9  x = np.dot(rng.rand(2, 2), rng.randn(2, 200)).T
10 # 绘制原始数据的散点图
11 plt.scatter(x[:, 0], x[:, 1])
12 # 确保横纵坐标轴的刻度比例一致
13 plt.axis('equal')
14
15 # PCA 分析
16 from sklearn.decomposition import PCA
17 # 创建一个 PCA 对象，指定保留 2 个主成分
18 pca = PCA(n_components=2)
19 # 对原始数据x进行拟合，计算主成分相关参数
20 pca.fit(x)
21 # 打印主成分向量
22 print(pca.components_)
23 # 打印每个主成分所解释的方差
24 print(pca.explained_variance_)
25
26 # 函数用于绘制向量
27
28 def draw_vector(v0, v1, ax=None):
29     ax = ax or plt.gca()
30     # noinspection SpellCheckingInspection
31     arrowprops=dict(arrowstyle='->',
32                     linewidth=2,
33                     shrinkA=0, shrinkB=0)
34     ax.annotate("", v1, v0, arrowprops=arrowprops)
35
36 # 重新绘制原始数据散点图，并根据主成分解释的方差和主成分向量绘制主成分向量，展示数据的主要变化方向
37 # plot data
38 plt.scatter(x[:, 0], x[:, 1], alpha=0.2)
```

```

39 for length, vector in zip(pca.explained_variance_, pca.components_):
40     v = vector * 3 * np.sqrt(length)
41     draw_vector(pca.mean_, pca.mean_ + v)
42 plt.axis('equal')
43
44 # 创建一个只保留 1 个主成分的 PCA 对象并拟合数据
45 pca = PCA(n_components=1)
46 # 拟合
47 pca.fit(X)
48 # 将原始数据X转换到主成分空间，实现降维
49 X_pca = pca.transform(X)
50 # 打印原始数据和降维后数据的形状
51 print("original shape: ", X.shape)
52 print("transformed shape:", X_pca.shape)
53
54 # 进行逆变换，将降维后的数据转换回原始特征空间
55 X_new = pca.inverse_transform(X_pca)
56 # 分别绘制原始数据和逆变换后数据的散点图，对比展示降维与逆变换的效果
57 plt.scatter(X[:, 0], X[:, 1], alpha=0.2)
58 plt.scatter(X_new[:, 0], X_new[:, 1], alpha=0.8)
59 plt.axis('equal')
60
61 plt.show()

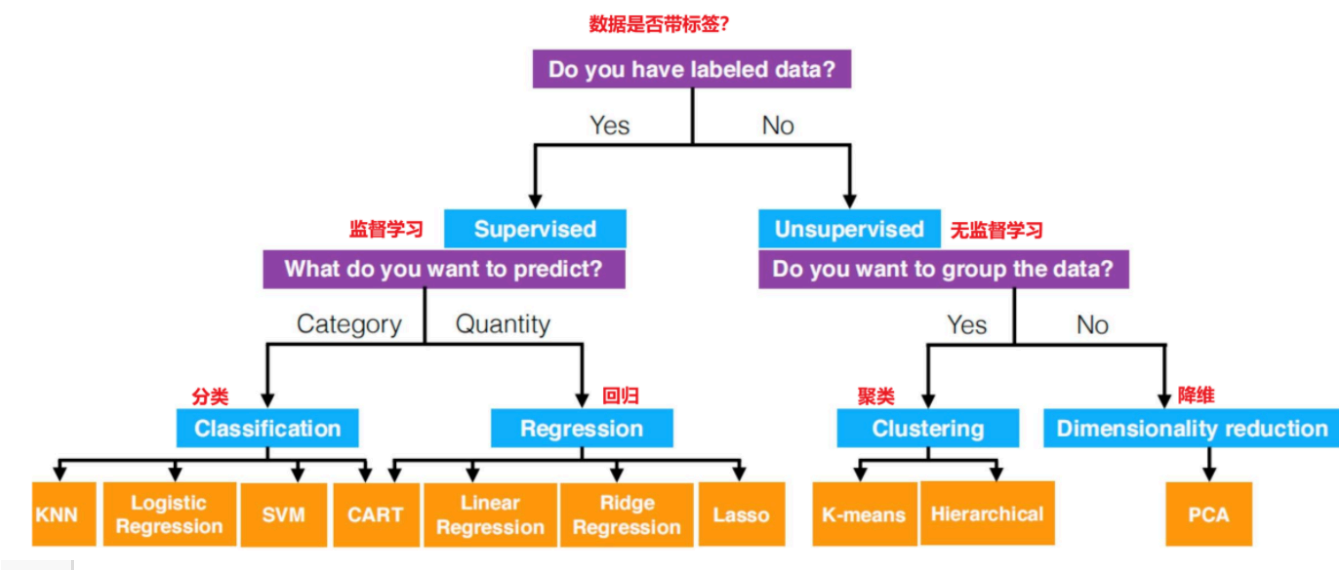
```



■ [Week 5]

[Gaussian Mixture Models or GMM](#) 和 [DBScan](#) 在这节课被提及

机器学习模式



KNN	K近邻算法
Logistic Regression	逻辑回归
SVM	支持向量机
CART	分类与回归树
Linear Regression	线性回归
Ridge Regression	岭回归
Lasso	套索回归
K-means	K均值算法
Hierarchical	层次聚类算法
PCA	主成分分析

无监督学习

一组用于处理仅包含特征/输入但没有响应变量的数据的统计工具。

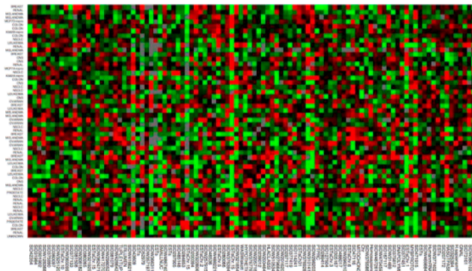
换句话说，我们有特征数据 X ，但没有标签 y 。

目标：发现数据中有趣的模式或特性。例如，用于可视化或解读高维数据。

示例

给定 n 位乳腺癌患者的组织样本，识别出乳腺癌的未知亚型。

基因表达实验会产生数千个变量。使用更少的一组特征来呈现数据，以便进行可视化和分析。



两种方法

1. 聚类分析

- 用于识别样本中的同质子组。

2. 降维

- 用于找到一种低维表示，以便描述数据特征和进行数据可视化。 PCA!

高斯混合模型（GMM）的标准构建

K个高斯分布的线性叠加

- μ_k : 均值 Mean
- σ_k : 协方差 Covariance
- 概率公式为: $p(\mathbf{x}_i) = \sum_{k=1}^K \underbrace{\pi_k}_{p(k)} \underbrace{\mathcal{N}(\mathbf{x}_i | \mu_k, \sigma_k)}_{p(\mathbf{x}_i | k)}, i = 1, \dots, N$
- 这个被称为高斯混合 (GM)。混合系数 π_k 满足: $\sum_{k=1}^K \pi_k = 1, \quad 0 \leq \pi_k \leq 1$

解释: 密度函数 $p(\mathbf{x} | k) = \mathcal{N}(\mathbf{x} | \mu_k, \sigma_k)$ 是在选择了第 k 个分量的情况下, $\mathbf{x}$ 的概率。选择第 k 个分量的概率由先验概率 $p(k)$ 给出。

例如, 考虑以下高斯混合模型:

$$p(x) = \underbrace{0.3}_{\pi_1} \mathcal{N}\left(x \middle| \begin{pmatrix} 4 \\ 4.5 \end{pmatrix}, \begin{pmatrix} 1.2 & 0.6 \\ 0.6 & 0.5 \end{pmatrix}\right) + \underbrace{0.5}_{\pi_2} \mathcal{N}\left(x \middle| \begin{pmatrix} 8 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right) + \underbrace{0.2}_{\pi_3} \mathcal{N}\left(x \middle| \begin{pmatrix} 9 \\ 8 \end{pmatrix}, \begin{pmatrix} 0.6 & 0.5 \\ 0.5 & 1.5 \end{pmatrix}\right)$$

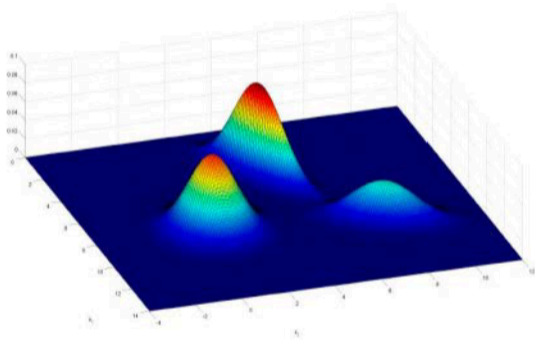


Figure: Probability density function.

概率密度函数

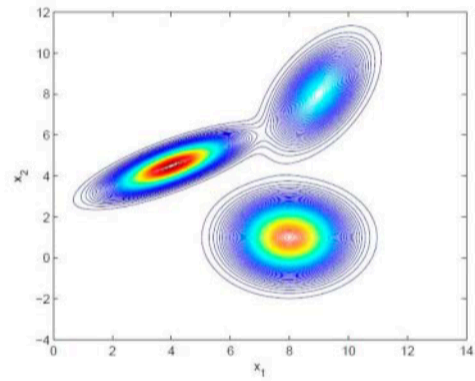


Figure: Contour plot.

等高线模型

高斯混合 (GM) 分布的形式由参数 π , μ 和 σ 决定。获取这些参数的一种方法是通过最大似然法。

给定 N 个观测值 $\{x_n\}_{n=1}^N$ ，对数似然函数为

$$\ln p(X; \pi_{1:K}, \mu_{1:K}, \sigma_{1:K}) = \sum_{n=1}^N \ln \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_n | \mu_k, \sigma_k) \right)$$

由于对数函数内部存在求和，所以不存在封闭形式的解。

这个问题可以使用期望最大化 (EM) 算法分解为两个简单的问题。

似然函数(Likelihood function)达到最大值时需满足的条件

对 $\ln p(x|\pi, \mu, \sigma)$ 关于 μ_k 求导：

$$\frac{d}{d\mu_k} [\ln p(x|\pi, \mu, \sigma)] = 0 \Rightarrow 0 = - \sum_{n=1}^N \underbrace{\frac{\pi_k \mathcal{N}(x_n | \mu_k, \sigma_k)}{\sum_j \pi_j \mathcal{N}(x_n | \mu_j, \sigma_j)}}_{\gamma(z_{nk})} \sigma_k (x_n - \mu_k)$$

由此可得： $\Rightarrow \mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n$

对 $\ln p(x|\pi, \mu, \sigma)$ 关于 σ_k 求导：

$$\frac{d}{d\sigma_k} [\ln p(x|\pi, \mu, \sigma)] = 0 \Rightarrow \sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k)(x_n - \mu_k)^T$$

使用拉格朗日乘数法，对 $\ln p(x|\pi, \mu, \sigma)$ 关于 π_k 求最大值，可得：

$$\pi_k = \frac{N_k}{N}, \text{ 其中 } N_k = \sum_{n=1}^N \gamma(z_{nk})$$

算法1 用于高斯混合模型的期望最大化 (EM) 算法

Algorithm 1 EM for Gaussian mixtures

- 1: Initialize $\mu_k^1, \sigma_k^1, \pi_k^1$ and set $i = 1$.
- 2: **while** not converged **do**
- 3: Compute $\gamma(z_{nk})$. ▷ Expectation step
- 4: Compute $\mu_k^{i+1}; \pi_k^{i+1}; N_k; \sigma_k^{i+1}$. ▷ Maximization step
- 5: $i \leftarrow i + 1$.
- 6: **end while**

$$\gamma(z_{nk}) = \frac{\pi_k^i \mathcal{N}(x_n | \mu_k^i, \sigma_k^i)}{\sum_{j=1}^K \pi_j^i \mathcal{N}(x_n | \mu_j^i, \sigma_j^i)}, n = 1, \dots, N; k = 1, \dots, K$$

$$\mu_k^{i+1} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n,$$

$$\pi_k^{i+1} = \frac{N_k}{N}, \quad N_k = \sum_{n=1}^N \gamma(z_{nk}),$$

$$\sigma_k^{i+1} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k^{i+1}) (x_n - \mu_k^{i+1})^T.$$

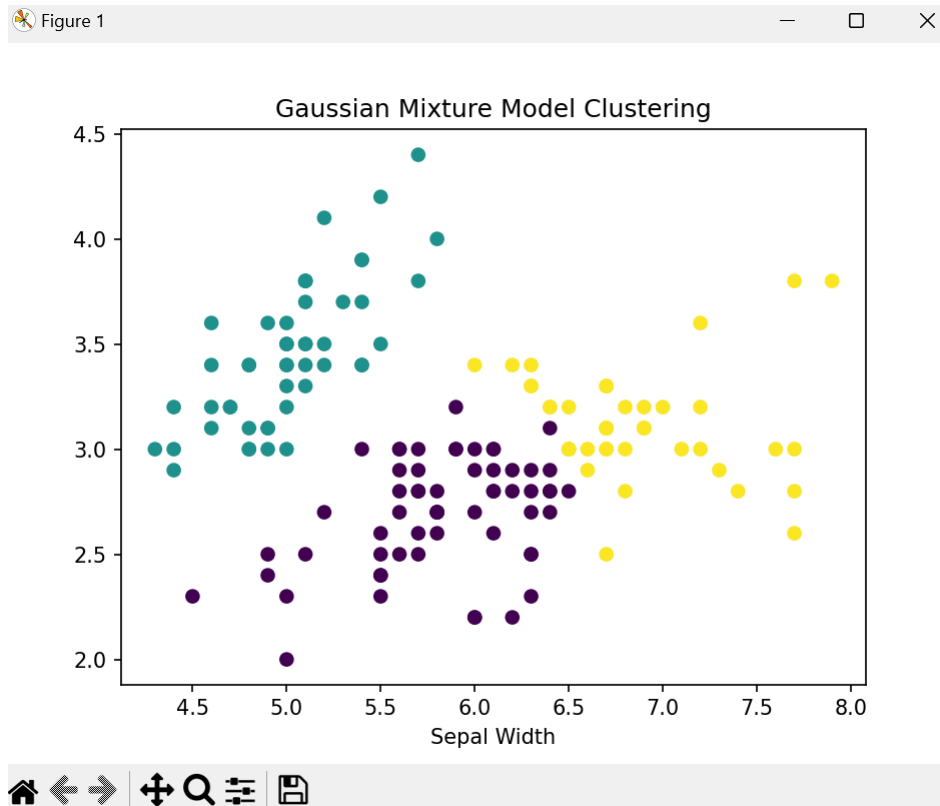
用Python实现 GMM

到目前为止，我们已经讨论了高斯混合模型的工作原理。下面我们以鸢尾花（Iris）数据集为例进行一个简单的演示，拟合一个具有3个簇的高斯混合模型（因为我们知道鸢尾花有3个品种）。

1. **数据集**：我们仅使用两个特征，即花萼长度和花萼宽度。
2. **拟合模型**：将数据拟合为3个高斯分布的混合。
3. **结果**：模型根据概率将每个数据点分配到一个簇中。收敛后，高斯混合模型将更新参数，以最佳地拟合数据。

数据库可以在 [Iris - UCI Machine Learning Repository](https://archive.ics.uci.edu/ml/datasets/Iris) 找到

```
1 from sklearn.mixture import GaussianMixture
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import load_iris
4
5 iris = load_iris()
6 x = iris.data[:, :2]
7 print(iris.data) # [x, y] x ↓ y →
8
9 gmm = GaussianMixture(n_components=3, covariance_type='full')
10 gmm.fit(x)
11
12 labels = gmm.predict(x)
13
14 plt.scatter(x[:, 0], x[:, 1], c=labels, cmap='viridis')
15 plt.xlabel('Sepal Length')
16 plt.ylabel('Sepal width')
17 plt.title('Gaussian Mixture Model Clustering')
```



这张散点图展示了使用高斯混合模型（GMM）对数据进行聚类的结果。数据点代表了基于花萼长度和花萼宽度对花卉品种的测量值。这些点被分为三个簇，分别用不同的颜色（黄色、紫色和绿色）表示，这表明高斯混合模型在数据中识别出了三个不同的组别。

GMM 优点

1. **灵活的簇形状**：与假设簇为球形的K均值算法不同，高斯混合模型可以对任意形状的簇进行建模。
2. **软分配**：高斯混合模型为每个数据点分配属于每个簇的概率，而K均值算法将每个点确切地分配到一个簇中。
3. **处理重叠数据**：当簇之间存在重叠或密度不同时，高斯混合模型表现良好。由于它使用概率分布，因此可以将一个点以不同的概率分配到多个簇中。

GMM 局限性

1. **计算复杂度**：高斯混合模型的计算成本往往较高，尤其是对于大型数据集，因为它需要诸如期望最大化（EM）算法这样的迭代过程来估计参数。
2. **选择簇的数量**：与其他聚类方法一样，高斯混合模型需要事先指定簇的数量。不过，像贝叶斯信息准则（BIC）和赤池信息准则（AIC）这样的方法可以帮助根据数据选择最优的簇数。

聚类分析之 DBScan

一组用于在数据集中寻找子组的方法...**聚类分析**！

- 同一组内的观测值应具有**共同的特征**，而不同组之间的观测值则存在差异。
- **分组**是由数据本身的属性决定的，这与**分类**有所不同。



[图]左侧：原始数据 右侧：聚类数据

基于质心的聚类（Centroid-based Clustering）

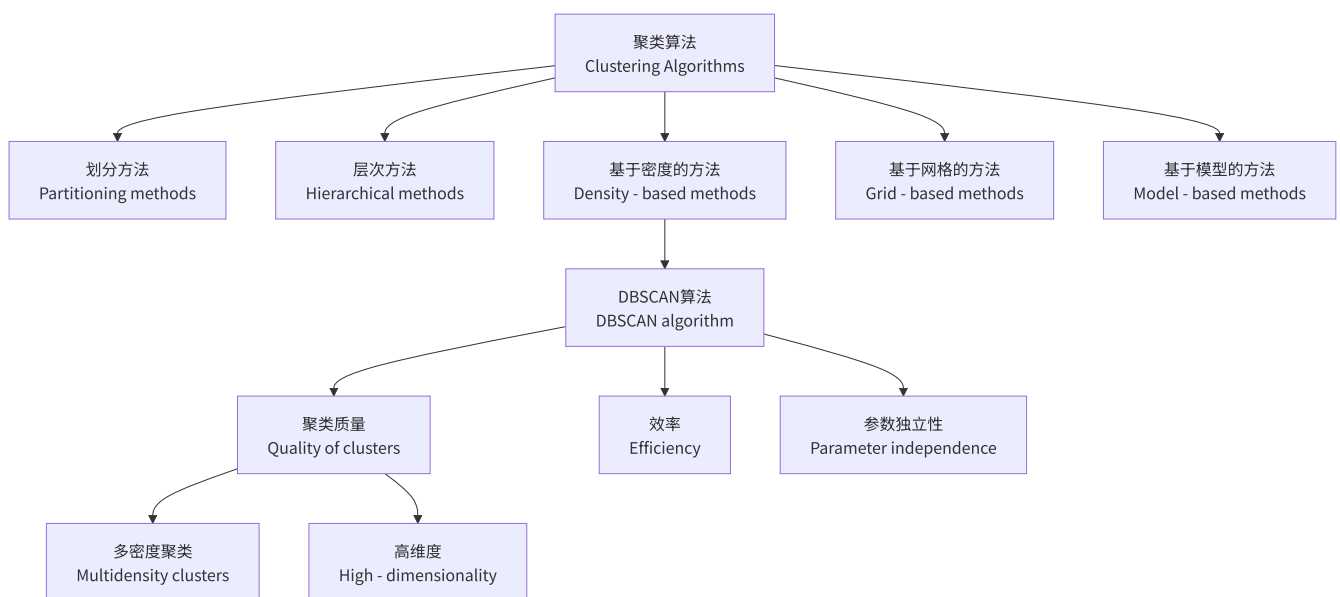
层次聚类（Hierarchical Clustering）

基于模型的聚类（Model-based clustering）

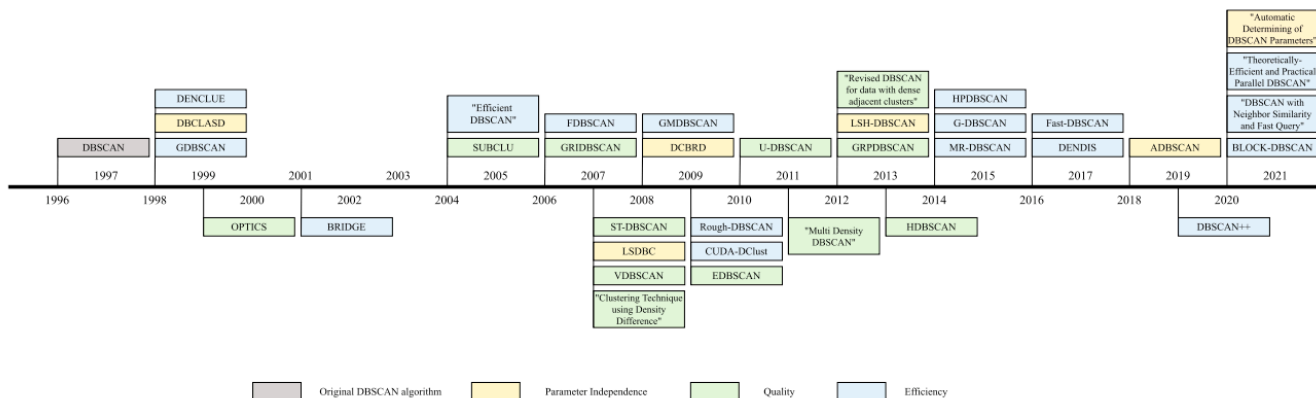
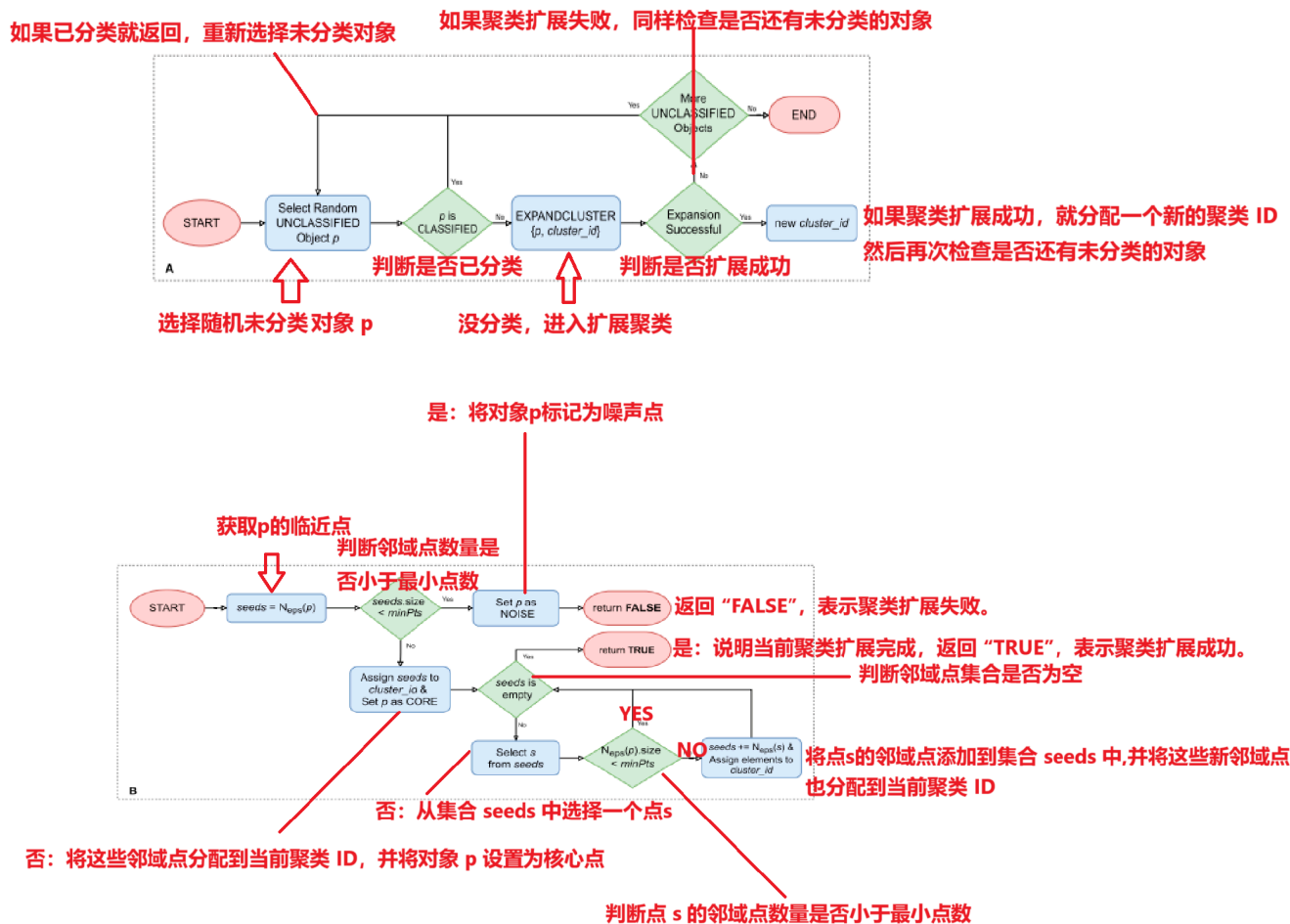
- 每个簇（Cluster）由一个**参数化分布**表示
- 数据集是多种分布的混合

硬聚类、软聚类、模糊聚类（Hard, Soft, Fuzzy Clustering）

- 硬聚类：观测值被划分到不同的明确簇中
- 软聚类：观测值可能属于多个簇



其中，DBScan全称 **Density-Based Spatial Clustering**，就是**基于密度的空间聚类**...



用 Python 实现 DBScan

[DBScan Clustering IRIS \(kaggle.com\)](https://www.kaggle.com/competitions/dbscan-clustering-iris)

Figure 1

```
1 import numpy as np
2 import pandas as pd
3
4 from sklearn import datasets
5 from sklearn.cluster import DBSCAN
6
7 import matplotlib.pyplot as plt
8 import seaborn as sns
```

```

9
10 iris = datasets.load_iris()
11
12 data_y = pd.DataFrame(iris.target)
13 data_y.columns = ['original_label']
14
15 data_x = pd.DataFrame(iris.data)
16 data_x.columns = ['Sepal length', 'Sepal width', 'Petal length', 'Petal width']
17
18 train = pd.concat([data_x, data_y], axis = 1)
19 train.head()
20
21 sns.color_palette("pastel")
22 sns.pairplot(train, hue='original_label')
23 plt.show()

```

Figure 2

```

1 TRAIN_LABEL_COL = 'original_label'
2 TRAIN_FEATURE = [col for col in train.columns if col != TRAIN_LABEL_COL]
3 X = train[TRAIN_FEATURE]
4 y = train[TRAIN_LABEL_COL]
5
6 dbscan = DBSCAN(eps=0.5, min_samples=5)
7 y_pred = dbscan.fit_predict(X)
8
9 y_pred_df = pd.DataFrame(y_pred)
10 y_pred_df.columns = ['pred_label']
11 val = pd.concat([X, y_pred_df], axis=1)
12
13 sns.color_palette("pastel")
14 sns.pairplot(val, hue='pred_label')
15 plt.show()
16
17 print(val)

```

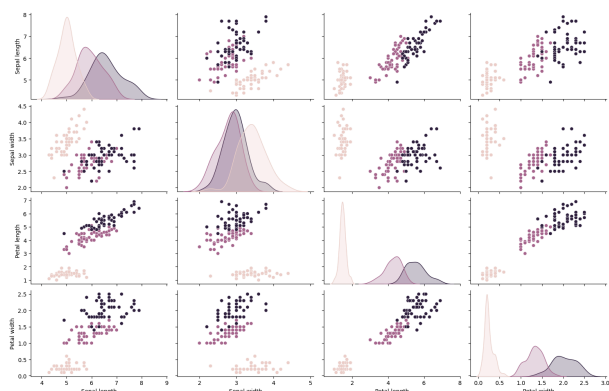


FIGURE 1

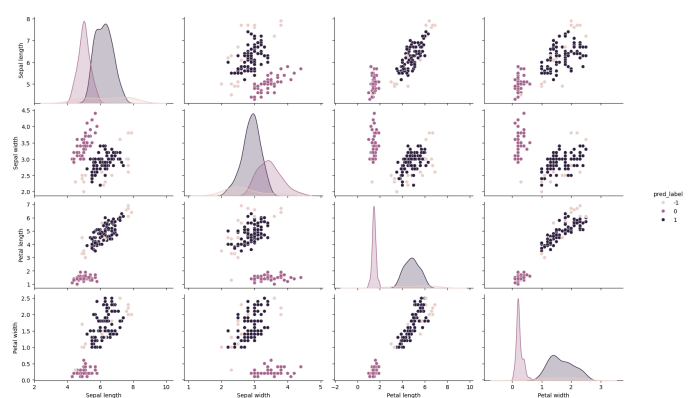
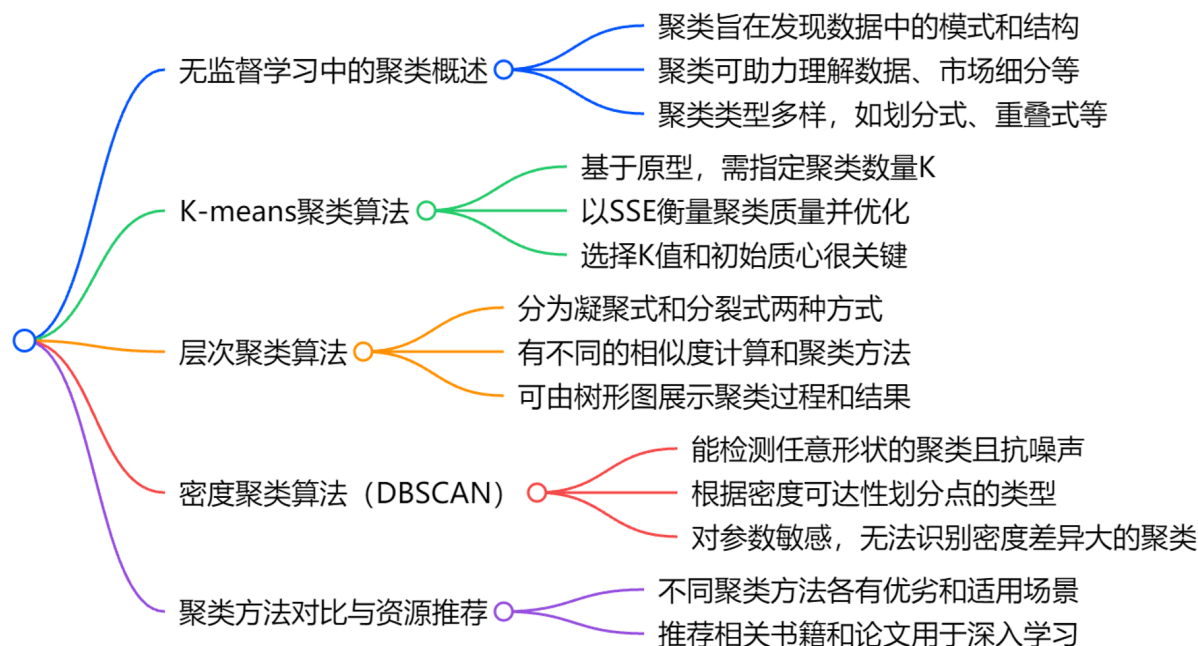


FIGURE 2

■ [Week 6]



[KMeans](#) 和 [Hierarchical](#) 聚类在本节提及

给定一组**无标签的数据点**，在数据中寻找模式或结构

聚类 Clustering：自动将数据点 / 项目分组为“相似”或“相关”的点组

主要挑战.....

- 如何衡量相似性？
- 理想的聚类数量是多少？是几个较大的聚类，还是更多较小的聚类？

我们有一些聚类的方法

划分式 Partitional

- 将很多点划分为不重叠的子集
- 每个点只属于一个子集

重叠式 Overlapping

- 将项目集划分为可能重叠的子集
- 每个项目可以同时属于多个子集

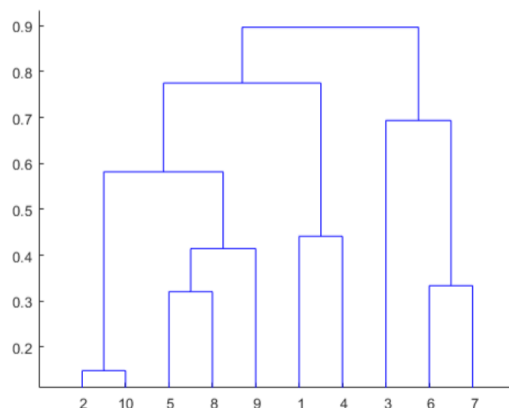
模糊式 Fuzzy

- 每个项目都以一个介于0（完全不属于）和1（完全属于）之间的隶属度权重，属于每个聚类。
- 常见的约束条件：每个项目的权重之和应为1。
- 转换为划分式聚类：将每个项目分配到你隶属度权重最高的聚类中。

层次式 Hierarchical

- 一组嵌套的聚类，其中一个较大的聚类可以包含较小的聚类。
- 以树状图（**dendrogram**）的形式组织：叶节点是包含单个项目的单例聚类，每个中间节点是其子聚类的并集。

树状图 Dendrogram 示例



- 一系列划分式聚类——在树状图的某一层次进行切割，即可得到一个划分式聚类。

完全聚类与部分聚类

- 完全聚类：将每个数据项分配到一个或多个簇中。
- 部分聚类：可能不会将某些数据项分配到任何簇中

（例如，离群点，即与其他任何数据项都不够相似的数据项）。

基于原型的（聚类方法） Prototype-based

- 每个簇由一个原型（质心或中心点）定义，即簇中最具代表性的点。
- 一个簇是这样一组数据项的集合：其中每个数据项与该簇的原型相比，与其他任何簇的原型相比距离更近（相似度更高）。
- 示例方法：**K - 均值算法**

基于密度的（聚类方法） Density-based

- 假设数据项分布在一个空间中，在该空间里“相似”的数据项彼此距离较近（例如，特征空间）。
- 一个簇是数据项的密集区域，且该区域被低密度区域包围。
- 示例方法：**DBSCAN（密度峰值聚类算法）**

基于图的（聚类方法）

- 假设数据项以图或网络的形式表示，其中数据项为节点，“相似”的数据项通过边相连。
- 一个簇是一组节点，这些节点内部的连接比它们与网络中其他部分的连接更多且 / 或更强。
- 在网络中也被称为“社区结构 community structure”。
- 示例方法：Girvan 和 Newman提出的算法

[1] K-Mean

- 基于原型的划分技术
- 找到用户指定数量的簇（K）
- 每个簇由其质心数据项表示

- 已有一些扩展方法，无需将簇的数量作为输入

随机初始化 K 个聚类中心 $\mu_1, \mu_2, \dots, \mu_K \in \mathbb{R}^n$

Repeat {

 对于 $i = 1$ 到 m

// 聚类分配

$c^{(i)} :=$ 距离 $x^{(i)}$ 最近的聚类中心（从 1 到 K ）的索引

// 移动中心

 for $k = 1$ to K

$\mu_k :=$ 分配到聚类 k 的点的平均值（均值）

}

优化K-Mean

考虑欧几里得空间中的数据点

一种衡量聚类质量的指标：误差平方和（SSE）

- 每个数据点的误差：该点到其最近质心的欧几里得距离
- SSE：每个点的误差平方的总和
- 如果一个簇的质心是该簇中所有数据点的平均值，那么SSE将达到最小

K - 均值算法的步骤是使SSE最小化（找到一个局部最小值）

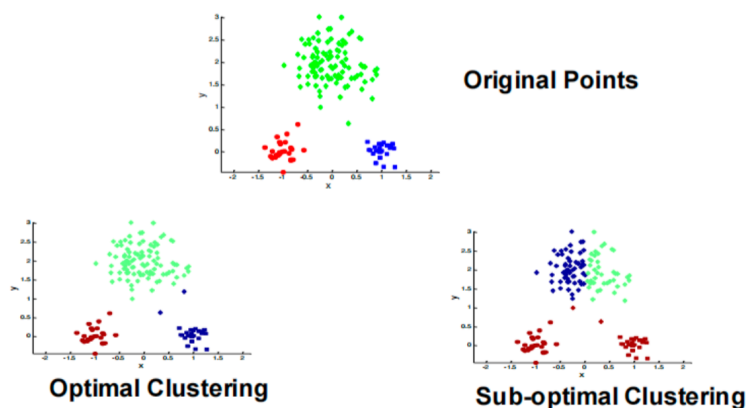
选择 K 值

根据特定问题领域中关于合适簇数量的领域知识（来确定簇的数量）。

或者，基于某种衡量聚类质量的指标，例如，尝试不同的 K 值，并选择使误差平方和（SSE）最小的那个 K 值。

选择初始质心

- 可以随机选择初始质心，但这可能会导致聚类效果不佳。
- 进行多次运行，每次使用一组不同的随机选择的初始质心，然后选择能使误差平方和（SSE）最小的那个配置。
- 利用领域知识来选择质心。例如，在对搜索结果进行聚类时，针对查询的每个方面选择一个相关的搜索结果。



数据项之间的相似性/接近度

- 数据项之间相似性/接近度的度量取决于问题领域。
- 在算法执行过程中，这种度量会被多次使用，因此需要高效。
- 示例
 - 欧几里得空间中的点→欧几里得距离
 - 文本文档→词向量之间的余弦相似度

通过后处理降低误差平方和（SSE）

- 找到更多的簇会降低SSE，但有时我们希望在增加簇数量的情况下改善SSE。
- K - 均值算法只能找到局部最小值；寻找另一个具有更低SSE的“邻近”聚类（如果存在的话）。

使用的技术

- **拆分一个簇**：例如，拆分误差平方和（SSE）最高的簇，或者拆分在选定特征上标准差最高的簇。
- **合并两个簇**：例如，合并质心距离最近的两个簇。

K-Mean 算法的已知问题

对异常值敏感，因为异常值会改变簇的分布。

- 一种解决方案：K-中心点算法（K-Medoids）。该算法不采用簇内点的均值，而是使用簇中最靠近中心位置的点（中心点）。

检测到的簇形状通常是球形的；在检测任意形状的簇时存在困难。

[2] 层次聚类

自下而上或凝聚式聚类

Agglomerative Clustering

- 一开始将每个数据点视为一个单独的簇。
- 如果相似度足够高，就不断合并簇。
- 直到所有点都合并为一个簇为止。

自上而下或分裂式聚类

Divisive Clustering

- 从所有数据点都在一个簇开始。
- 如果两个子部分之间的相似度较低，就迭代地将簇分裂成更小的子簇。

基本凝聚式层次聚类算法

Agglomerative Clustering Alg.

- 初始时，将每个数据项视为一个单独的簇。
- 计算簇与簇之间的邻近度/相似度矩阵。
- 重复以下步骤：
 - 合并距离最近/相似度最高的两个簇。
 - 更新邻近度矩阵，以反映新簇与其他簇之间的邻近度。
- 直到只剩下一个簇为止。

簇间的邻近度/相似度

- **两个簇之间的最小相似度**：两个簇中距离最近（相似度最高）的两点（每个簇各取一点）之间的邻近度（相似度）（即最小成对距离）。
- **两个簇之间的最大相似度**：两个簇中距离最远的两点（每个簇各取一点）之间的邻近度（即最大成对距离）。
- **组平均相似度**：两个簇中所有点对（每个簇各取一点组成的点对）的平均成对邻近度。

层次聚类的类型

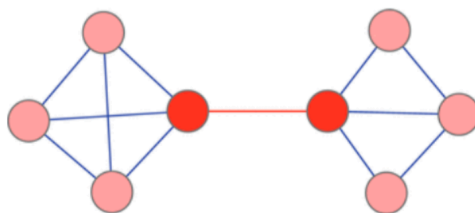
- **全连接（完全连锁）** Complete Linkage
 - 每一步合并 具有最小“最大相似度” 的两个簇。
- **单连接（单连锁）** Single Likage
 - 每一步合并 具有最小“最小相似度” 的两个簇。

一种基于图的层次聚类算法

- 一个簇是一组节点，这些节点内部的连接比它们与网络中其他部分的连接更多且 / 或更强。
- 图/网络中的簇，也称为社区结构。
- Girvan 和 Newman提出的算法： *Communi structure in social and biological networks*, PNAS 2002

Girvan - Newman算法

- 关注那些最处于簇“之间” 的边/连接。
- 一条边 e 的边介数中心性：在所有节点对之间的最短路径中，经过边 e 的路径所占的比例。



边介数中心性

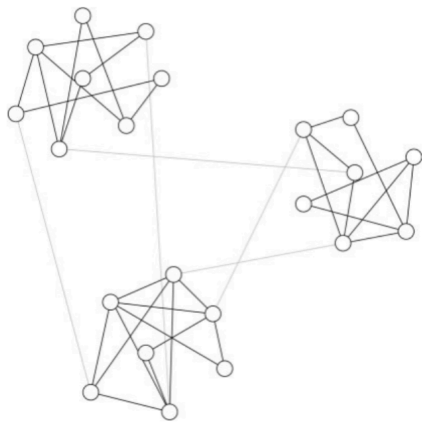
定义

从结构上定位“连接紧密”的边。

如果一条边位于许多最短路径上。

$$BC(e) = \sum_{v,w \in V} \frac{b_{vw}(e)}{b_{vw}}$$

- $b_{vw}(e)$ = 从节点 v 到节点 w 经过边 e 的最短路径数量
- b_{vw} = 从节点 v 到节点 w 的最短路径总数
- 簇/社区之间的边往往具有较高的介数中心性。
- 逐步移除具有高介数中心性的边，以使各个簇相互分离。



步骤：

1. 计算所有边的介数中心性 **Centrality**。
2. 删除介数中心性最高的边。
3. 重新计算因删除操作而受到影响的所有边的介数中心性。
4. 重复步骤2和步骤3，直到没有边剩余。

最终生成一棵层次聚类树（树形图）。

[3] 基于密度的聚类方法

Density based clustering methods

- 定位高密度区域，这些区域被低密度区域相互分隔开来



[4] DBSCAN 密度聚类算法

- DBSCAN：基于密度的带噪声空间聚类应用算法
 - 由埃斯特（Ester）等人于1996 年在国际知识发现和数据挖掘会议（SIGKDD）上提出。
 - 首个用于检测基于密度的簇的算法。
- 优点（例如，相对于K - 均值算法）
 - 可以检测任意形状的簇（而K - 均值算法检测到的簇通常是球形的）。

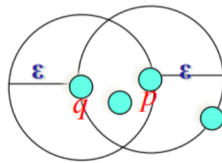
- 对异常值具有鲁棒性（robustness）。

“robustness”指的是系统、算法或模型等在面临各种干扰、变化（如数据噪声、参数波动、环境改变等）的情况下，仍能保持其性能稳定、正常运行且不出故障或显著退化的能力。

基本思想

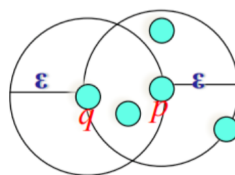
- 对于簇中的任意一点，该点周围的**局部点密度**必须超过某个阈值。
- 一个簇中的点集在空间上是**连通的**。
- 点 p 处的局部点密度由两个参数定义：
 - ε ：点 p 邻域的半径， $N_\varepsilon(p) := \{q \text{ in data set} \mid \text{dist}(p, q) \leq \varepsilon\}$ 。
 - $MinPts$ ：给定邻域 $N_\varepsilon(p)$ 中的最少点数。

点的邻域



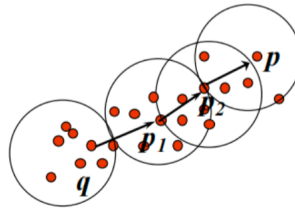
- 点 p 的 ε -邻域：距离点 p 半径为 ε 范围内的点，即 $N_\varepsilon(p) = \{q \text{ 在数据集中} \mid \text{dist}(p, q) \leq \varepsilon\}$ 。
- “高密度”：如果一个点的 ε -邻域中至少包含 $MinPts$ 个点。
 - p 的 ε -邻域
 - q 的 ε -邻域
 - 当 $MinPts = 4$ 时， p 的密度为“高”
 - 当 $MinPts = 4$ 时， q 的密度为“低”

密度可达性



$MinPts = 4$

- 直接密度可达**：如果点 p 是核心点，且点 q 在点 p 的 ε -邻域内，那么点 q 从对象 p 直接密度可达。
 - q 从 p 直接密度可达
 - p 不从 q 直接密度可达
 - 密度可达性不是对称的
 （其中 $MinPts = 4$ ）
- 密度可达性可以是直接的或间接的



- 点 p 从 p_2 直接密度可达
- p_2 从 p_1 直接密度可达
- p_1 从 q 直接密度可达
- $p \leftarrow p_2 \leftarrow p_1 \leftarrow q$ 形成一条链

(其中 $MinPts = 7$)

- p 从 q (间接) 密度可达
- q 不从 p 密度可达

DBSCAN 算法

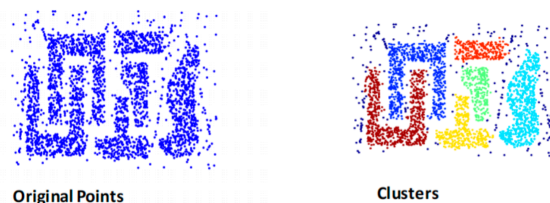
```

1  输入：数据集  $D$ 
2  参数： $\epsilon$ ,  $MinPts$ 
3  for each  $p$  in  $D$ 
4      if  $p$  是核心点且未被处理过
5           $C = \{ \text{所有从 } p \text{ 密度可达的点} \}$ 
6          将  $C$  中的所有点标记为已处理
7          将  $C$  作为一个簇输出
8      else
9          将  $p$  标记为离群点
10     end if
11 end for

```

- 任意选择一个点 p 。
- 根据参数 ϵ 和 $MinPts$ ，检索从点 p 密度可达的所有点。
- 如果 p 是核心点，则形成一个簇。
- 如果 p 是边界点，那么从 p 没有密度可达的点，DBSCAN算法会访问数据库中的下一个点。
- 继续这个过程，直到所有的点都被处理完（每个点都被标记为核心点、边界点或离群点）。

什么时候DBSCAN会有比较好的表现？



- 抗噪声/离群点（注：属于部分聚类）
- 能够处理不同形状和大小的簇
- 可自动识别簇的数量
- 无法识别密度各异的簇

- 对参数敏感

聚类相关资源（可在网上免费获取）

- 《数据聚类：算法与应用》

- 作者：查鲁·阿加瓦尔（Charu Aggarwal）、钱丹·雷迪（Chandan Reddy）

Data Clustering: Algorithms and Applications

- Book by Charu Aggarwal, Chandan Reddy

- 《图中的社区检测》

- 综述论文，作者：桑托·福尔图纳托（Santo Fortunato）

Community Detection in graphs

- Survey paper by Santo Fortunato

■ [Week 7]

[Supervised Learning](#), [SVM \(支持向量机\)](#), [Naive-Bayes-Classifier \(朴素贝叶斯\)](#)

[1] 监督学习

什么是监督学习

监督学习是一种机器学习技术，它使用**带标签的数据集**来训练人工智能算法模型，以识别输入特征与输出之间的潜在模式和关系。学习过程的目标是创建一个模型，该模型能够正确地预测新数据的输出。

带标签的数据由**示例数据点+正确的输出或答案**组成。当输入数据被输入到机器学习算法中时，算法会**调整其权重**，直到模型得到适当的拟合。带标签的训练数据明确地教会模型识别特征和数据标签之间的关系。

监督式机器学习有助于组织大规模解决各种现实世界问题，例如对垃圾邮件进行分类或预测股票价格。它可用于构建高度准确的机器学习模型。

监督学习如何运作

监督学习使用带标签的训练数据集来理解输入和输出数据之间的关系。数据科学家**手动创建包含输入数据及相应标签的训练数据集 (Data Set)**。监督学习对模型进行训练，以便在实际应用场景中对新的输入数据给出正确的输出。

在训练过程中，模型的算法处理大量数据集，以探寻输入和输出之间可能存在的相关性。然后，使用测试数据评估模型性能，以确定其训练是否成功。**交叉验证 (Cross-Validation)** 是使用数据集的不同部分来测试模型的过程。

梯度下降 (Gradient Decent) 系列算法，包括**随机梯度下降 (SGD-Stochastic Gradient Decent)**，是训练神经网络和其他机器学习模型时最常用的优化算法（即学习算法）。模型的优化算法通过**损失函数 (Loss Function)** 评估准确性，损失函数是一种衡量模型预测值与实际值之间差异的公式。

损失函数的斜率，即梯度，是衡量模型性能的主要指标。优化算法沿梯度下降以最小化其值。在整个训练过程中，优化算法会更新模型的参数（即其运行规则或“设置”），以优化模型。

监督学习工作流程

一个典型的监督学习过程可能如下：

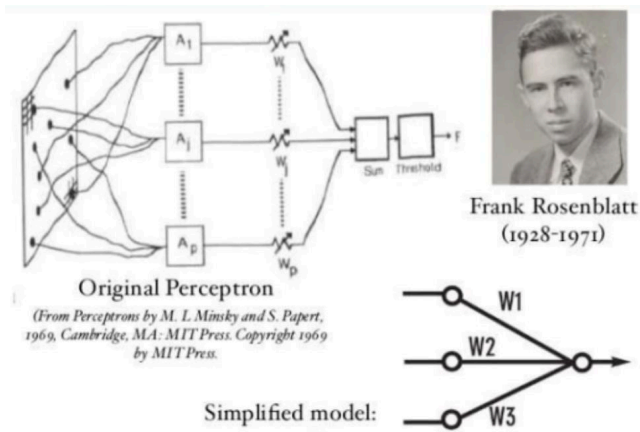
1. 确定训练模型的**训练数据类型**。该数据应与模型投入使用后要处理的预期输入数据相似。
2. 收集训练数据**并进行标注**，以创建**带标签的训练数据集**。训练数据**必须无数据偏差**，以避免由此产生的算法偏差和其他性能缺陷。
3. 创建三组数据：**训练数据、验证数据和测试数据**。（**训练、验证、测试集**）验证用于评估训练过程，以便进一步调整和优化，而测试用于评估最终模型。
4. 选择一种机器学习学习算法。
5. 将训练数据集输入所选算法。
6. 相应地对模型进行验证和测试。
7. 关注模型性能，并通过定期更新保持准确性。

监督学习在现实世界中的应用案例

- **垃圾邮件检测**：垃圾邮件检测是监督学习模型的另一个例子。通过使用监督分类算法，机构可以对数据库进行训练，使其识别新数据中的模式或异常情况，从而有效地对垃圾邮件和非垃圾邮件通信进行分类。
- **预测（Forecasting）**：回归模型擅长基于历史趋势进行预测，这使其适用于金融行业。企业还可以使用回归分析来预测库存需求、估算员工薪资，并避免潜在的供应链问题。
- **推荐引擎**：借助监督学习模型，内容提供商和在线市场可以分析客户的选择、偏好和购买行为，进而构建推荐引擎，提供更有可能促成交易的个性化推荐。

[2] 感知机

感知机（Perceptron）是一种神经网络单元（人工神经元），它通过进行特定计算来检测输入数据中的特征或商业智能信息。



感知机概述

感知机具有以下特征：

- 感知机是一种用于**单层二元线性分类器（Linear 2-Classification Task）**的监督学习算法。
- 最优权重系数可自动学习。
- 权重与输入特征相乘，以此决定神经元是否激发。
- 激活函数采用阶跃规则，检查加权函数的输出是否大于零。
- 绘制线性决策边界，以区分 +1 和 -1 这两个线性可分的类别。
- 如果输入信号的总和超过某个阈值，就会输出一个信号；否则，无输出。

感知机学习规则

- 感知机学习规则表明，算法会**自动学习最优权重系数**。
- 然后，将输入特征与这些权重相乘，以确定神经元是否激发。

感知机规则

- **净输入函数（Net input function）**
- **激活函数（Activation function）**

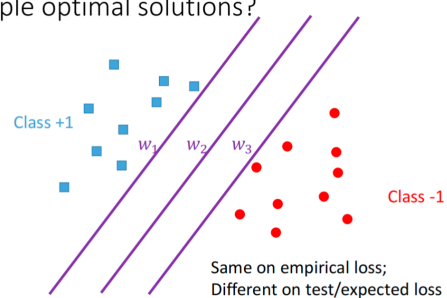
支持向量机 (SVM)

给定训练数据 $\{(x_i, y_i) : 1 \leq i \leq n\}$ ，它们独立同分布于分布 D 。假设 $y = \text{sign}(f_w(x)) = \text{sign}(w^T x)$ ：

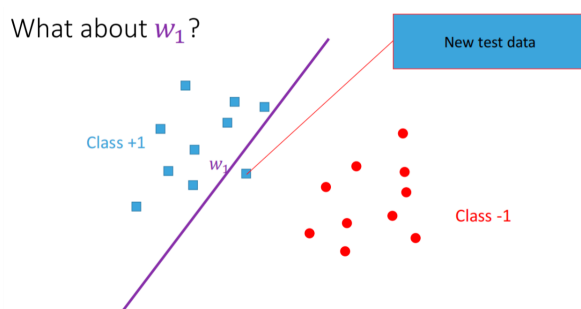
- 若 $w^T x > 0$ ，则 $y = +1$ 。
- 若 $w^T x < 0$ ，则 $y = -1$ 。

假设我们可以通过优化找到 w 。

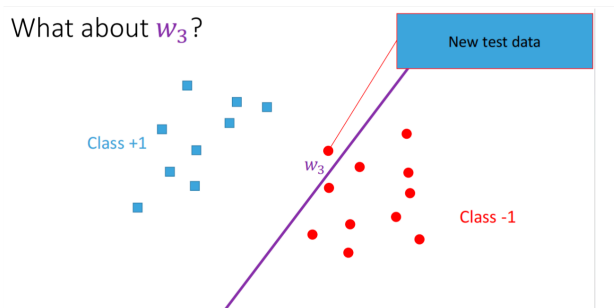
Multiple optimal solutions?



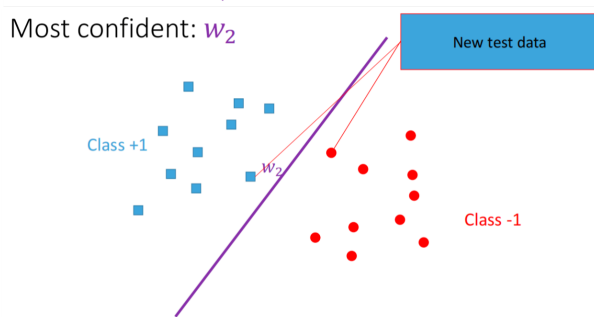
What about w_1 ?



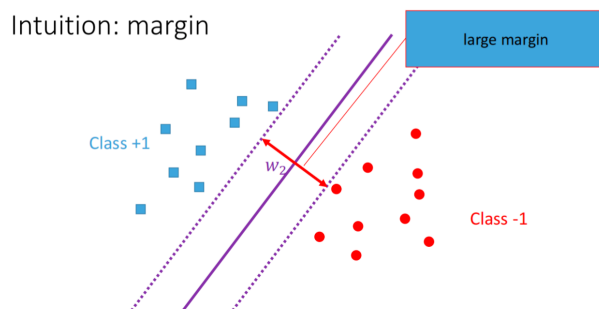
What about w_3 ?



Most confident: w_2



Intuition: margin



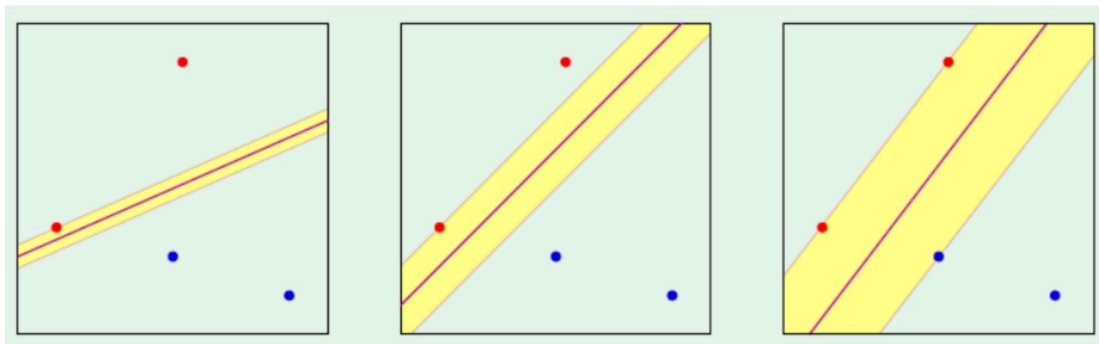
SVM 试图找到一个超平面，不仅能正确分类现有数据，还能使两类数据点到超平面的最小距离（即间隔 **Margin**）尽可能大。这样可以降低模型对训练数据的过拟合风险，提升在未知数据上的分类准确性。

间隔的直观理解

存在许多可能的分隔线。哪条分隔线是最佳的呢？间隔：从最近的数据点到分隔线的距离

间隔越大越好（保证损失函数不变情况下）→泛化能力越强

泛化能力（Generalization）指的是模型对未知数据的适应和预测能力。



这张图中第三章间隔是最大的，因此泛化能力最强

寻找决策边界

- 我们希望找到这样一个决策边界：它不仅能正确分类所有的数据点，还能使间隔最大化（泛化能力最强）。

假设是d维特征空间。

d维特征空间中的决策边界：一个（超）平面。我们假设数据是线性可分的；分隔平面不会接触任何数据点。

超平面

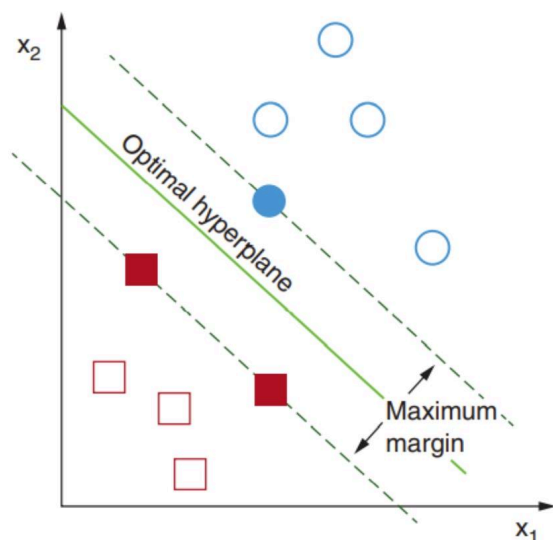
什么是超平面？简单来说就是比d维少1维。

例如二维数据超平面就是1维，一条线。一条线能够线性分割二维平面

例如三维数据超平面就是2维，一个面，一个面能够线性分割三维平面

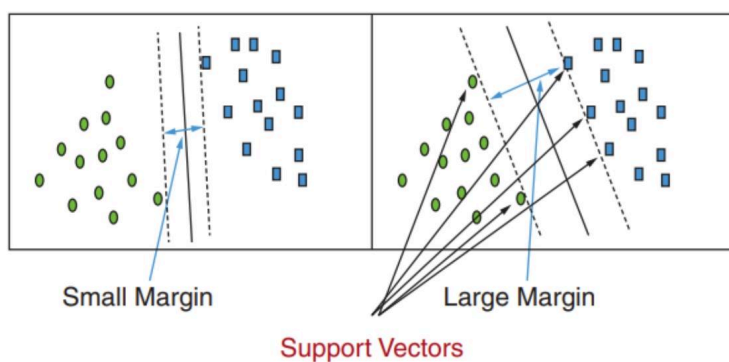
超平面可表示为 $WX + b = 0$ ，因此：

- 正类样本满足 $WX + b > +1$ 。
- 负类样本满足 $WX + b < -1$ 。
- 间隔的值为 $\frac{2}{\|W\|}$



支持向量

与超平面距离最小的样本被称为支持向量。我们希望最大化超平面与支持向量之间的距离。



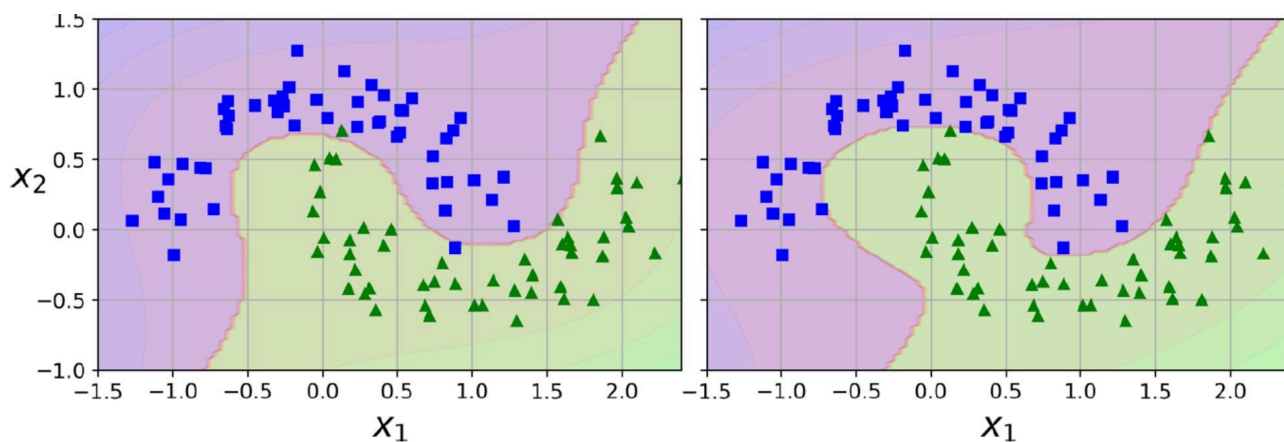
核函数

Kernel

在某些情况下，核函数被用于使原本线性不可分的情况变得线性可分。

多项式特征和核函数

可以添加第二个特征，例如 $x_2 = (x_1)^2$ 。也可以使用核函数来替代添加多项式特征。



在机器学习中，当处理线性不可分的数据时，一种方法是通过添加多项式特征，将数据映射到更高维的空间，使其变得线性可分，就像之前提到的添加 $x_2 = (x_1)^2$ 这样的特征。但这种方法会增加计算复杂度和特征维度。

而**多项式核函数提供了一种更巧妙的方式**，它在支持向量机等算法中应用，不需要显式地计算和添加新的多项式特征。核函数在低维空间中通过特定的数学运算（核函数计算），**隐式地在高维特征空间中进行内积运算，达到与添加多项式特征类似的效果**，即让原本线性不可分的数据在高维空间中能被线性分类，**同时避免了高维空间中复杂的计算和维度灾难问题。**

图中展示了在使用多项式核函数前后，对数据进行分类的情况，体现了核函数在处理非线性分类问题上的有效性。

[3] 贝叶斯分类器

- 一种用于解决**分类问题**的概率框架。
- 一种对属性集和类别变量之间的概率关系进行建模的方法。
 - **即使测试记录与某些训练记录具有相同的属性，也可能无法确定地预测其类别标签。**
 - 原因：数据存在噪声，或者存在某些未纳入分析的因素。

概率基础

- $P(A = a, C = c)$ ：随机变量 A 和 C 分别取值为 a 和 c 的**联合概率(Joint probability)**。
- $P(A = a|C = c)$ ：在 C 取值为 c 的条件下， A 取值为 a 的**条件概率(Conditional Probability)**。

$$P(C|A) = \frac{P(A, C)}{P(A)}$$
$$P(A|C) = \frac{P(A, C)}{P(C)}$$

贝叶斯定理

贝叶斯定理：

$$P(C|A) = \frac{P(A|C)P(C)}{P(A)}$$

$P(C)$ 被称为先验概率。

$P(C|A)$ 被称为后验概率。

贝叶斯定理示例

- 已知：
 - 医生知道脑膜炎导致脖子僵硬的概率为50%。

记为 $S|M$ ，也就是脖子僵硬 S ，但前提是脑膜炎 M 导致的
 - 任何患者患脑膜炎的先验概率是1/50,000。

记为 M
 - 任何患者脖子僵硬的先验概率是1/20。

记为 S
- 如果一位患者脖子僵硬（ S ），那么他/她患脑膜炎（ M ）的概率是多少？

$$P(M|S) = \frac{P(S|M)P(M)}{P(S)} = 0.0002$$

将每个属性和类别标签视为随机变量。

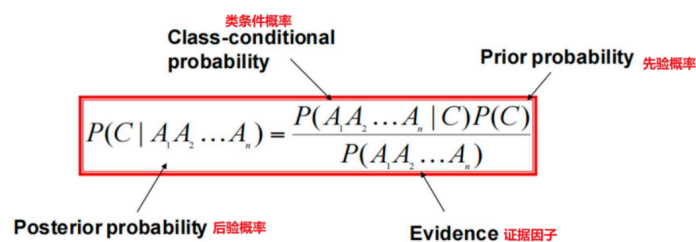
给定一条具有属性 $(A_1, A_2, \dots, A_n)$ 的记录：

- 目标是预测类别 C 。
- 具体来说，我们想要找到使 $P(C|A_1, A_2, \dots, A_n)$ 最大化的 C 值。

我们能否直接从数据中估计 $P(C|A_1, A_2, \dots, A_n)$ 呢？

方法：使用贝叶斯定理，为 C 的所有取值计算后验概率 $P(C|A_1, A_2, \dots, A_n)$

$$P(C|A_1 A_2 \dots A_n) = \frac{P(A_1 A_2 \dots A_n | C) P(C)}{P(A_1 A_2 \dots A_n)}$$



然后，我们的目标是：要找到使 $P(C|A_1, A_2, \dots, A_n)$ 最大化的 C 值。这就等同于找到 $P(A_1 A_2 \dots A_n | C) P(C)$ 最大化下的 C 值。

现在的目标变成：如何估计 $P(A_1 A_2 \dots A_n | C)$ 呢？

↓

朴素贝叶斯分类器

Naïve Bayes Classifier

假设类别 C 给定的情况下，所有属性 A_i 都条件独立

$$P(A_1 A_2 \dots A_n | C) = P(A_1 | C) P(A_2 | C) \dots P(A_n | C)$$

那么对于所有的 A_i 和 C_j 就可以估 $P(A_i | C_j)$ 了。

如果 $P(C_j) \prod P(A_i | C_j)$ 最大，那么新的数据点将被分类到 C_j 类别

条件独立性：基础

如果 $P(X|Y) = P(X)$ ，那么能证明 X 和 Y 相互独立。

也就是 Y 事件的发生不影响 X 事件

设 X 、 Y 、 Z 表示三组随机变量。

若满足 $P(X|Y, Z) = P(X|Z)$ ，则称在给定 Z 的条件下， X 中的变量与 Y 中的变量 **条件独立**。

比如 X 代表阅读能力， Y 代表手臂长度， Z 代表年龄。

如果我们不知道年龄 Z ，那么从概率和生活实际上看：

阅读能力 X 和手臂长度 Y 有一定关联，不相互独立，因为这个时候 $P(X|Y) \neq P(X)$

然而，当给定年龄 Z 时（年龄被控制时），手臂长度 Y 不再对阅读技能水平 X 的概率产生额外影响，即满足条件独立性公式 $P(X|Y, Z) = P(X|Z)$ 。也就是说，在已知年龄的条件下，仅根据手臂长度无法再对阅读技能水平的概率进行推断，所以手臂长度 Y 和阅读技能 X 在给定年龄的条件下是条件独立的。

若在给定 Z 的条件下， X 和 Y 条件独立：

条件概率的定义：

$$\begin{aligned}P(X|Y) &= \frac{P(XY)}{P(Y)} \\P(X, Y|Z) &= \frac{P(X, Y, Z)}{P(Z)} \\&= \frac{P(X, Y, Z)}{P(Y, Z)} \times \frac{P(Y, Z)}{P(Z)} \\&= P(X|Y, Z) \times P(Y|Z) \\&= P(X|Z) \times P(Y|Z)\end{aligned}$$

也就是

$$P(X, Y|Z) = P(X|Z) \times P(Y|Z)$$

朴素贝叶斯（NB）假设：

$$P(A_1, A_2, \dots, A_n|C) = P(A_1|C)P(A_2|C) \dots P(A_n|C)$$

How to Estimate Probabilities from Data?

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

● Class: $P(C) = N_c/N$

— e.g., $P(\text{No}) = 7/10$,
 $P(\text{Yes}) = 3/10$

● For discrete attributes:

$$P(A_i | C_k) = |A_{ik}| / N_{C_k}$$

— where $|A_{ik}|$ is number of instances having attribute A_i and belongs to class C_k

— Examples:

$$\begin{aligned}P(\text{Status}=\text{Married}|\text{No}) &= 4/7 \\P(\text{Refund}=\text{Yes}|\text{Yes}) &= 0\end{aligned}$$

[AI解释]

在数据分析和机器学习中，常常需要从给定的数据集中估计概率，以便用于分类等任务。这里介绍的方法主要是针对离散型数据来估计类别概率和离散属性的条件概率。

估计类别概率 $P(C)$

- **目的：**确定数据集中每个类别在总体中出现的可能性大小，得到类别先验概率，为后续基于贝叶斯理论的分类任务提供基础。
- **公式：** $P(C) = \frac{N_c}{N}$ 。其中， N 代表数据集中总的样本数量，它涵盖了所有不同类别的样本。 N_c 表示属于特定类别 C 的样本数量。通过这个比例关系，能衡量出某一类别在整个数据集所占的比重。
- **举例：**以图中表格数据为例，总共有 $N = 10$ 条记录（可通过逐行计数得到）。在“Evade”这个类别属性下，有“ No”和“ Yes”两个类别取值。其中“Evade”类别为“ No”的记录数 $N_{No} = 7$ 条，为“ Yes”的记录数 $N_{Yes} = 3$ 条。所以根据公式计算可得， $P(No) = \frac{N_{No}}{N} = \frac{7}{10}$ ， $P(Yes) = \frac{N_{Yes}}{N} = \frac{3}{10}$ 。这表明在该数据集中，样本属于“Evade”为“ No”类别的可能性更大。

估计离散属性的条件概率 $P(A_i|C_k)$

- **目的：**在已知样本属于特定类别的条件下，确定某个离散属性出现的概率。这对于构建贝叶斯分类器等模型至关重要，因为它能帮助衡量在某类别中不同属性取值的分布情况，进而用于预测新样本的类别。
- **公式：** $P(A_i|C_k) = \frac{|A_{ik}|}{N_{ck}}$ 。这里， N_{ck} 是类别 C_k 的样本数量，即数据集中属于特定类别 C_k 的样本总数。 $|A_{ik}|$ 表示在类别 C_k 中，属性 A_i 所出现的次数，也就是同时满足具有属性 A_i 且属于类别 C_k 的样本数量。通过这个公式，能得到在特定类别条件下，某个离散属性出现的条件概率。
- **举例：**
 - 计算 $P(\text{Status} = \text{Married}|\text{No})$ ，首先确定“Evade”为“ No”这一类别下的样本数量 $N_{No} = 7$ （即 $N_{ck} = 7$ ）。然后在这7条记录中，统计“Marital Status”（即属性 A_i ）为“Married”的记录数，发现有4条（即 $|A_{ik}| = 4$ ）。根据公式可得 $P(\text{Status} = \text{Married}|\text{No}) = \frac{|A_{ik}|}{N_{ck}} = \frac{4}{7}$ 。这意味着在“Evade”为“ No”的样本中，“Marital Status”为“Married”的概率是 $\frac{4}{7}$ 。
 - 计算 $P(\text{Refund} = \text{Yes}|\text{Yes})$ ，先找出“Evade”为“ Yes”的样本数量 $N_{Yes} = 3$ （即 $N_{ck} = 3$ ）。接着在这3条记录中，统计“Refund”为“ Yes”的记录数，发现是0条（即 $|A_{ik}| = 0$ ）。所以 $P(\text{Refund} = \text{Yes}|\text{Yes}) = \frac{|A_{ik}|}{N_{ck}} = 0$ ，即在“Evade”为“ Yes”的样本中，“Refund”为“ Yes”的概率为0。

[PPT] 解释

如何从数据中估计概率？

对于连续属性，有两种方法：

- **离散化：**将取值范围划分为若干区间。
 - 每个区间对应一个有序属性。
- **概率密度估计：**
 - 假设属性服从高斯/正态分布。
 - 利用数据估计分布的参数（例如均值和标准差）。
 - 一旦确定概率分布，就可以用它来估计条件概率 $P(A_i|c)$ 。

- 正态分布：

$$P(A_i|c_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} e^{-\frac{(A_i - \mu_{ij})^2}{2\sigma_{ij}^2}}$$

- 对于每一对 (A_i, c_j) 都有一个这样的概率值。

- 对于（收入，类别 = 未逃税）：

- 如果类别 = 未逃税

- 样本均值 = 110
- 样本方差 = 2975

$$P(\text{收入} = 120 | \text{未逃税}) = \frac{1}{\sqrt{2\pi(2975)}} e^{-\frac{(120 - 110)^2}{2(2975)}} = 0.0072$$

朴素贝叶斯分类器示例

- 给定测试记录：

$X = (\text{退款} = \text{否}, \text{已婚}, \text{收入} = 120K)$

- 训练数据：

Tid	退款	婚姻状况	应纳税收入	是否逃税
1	是	单身	125K	否
2	否	已婚	100K	否
3	否	单身	70K	否
4	是	已婚	120K	否
5	否	离异	95K	是
6	否	已婚	60K	否
7	是	离异	220K	否
8	否	单身	85K	是
9	否	已婚	75K	否
10	否	单身	90K	是

朴素贝叶斯分类器：

- $P(\text{退款} = \text{是} | \text{否}) = 3/7$
- $P(\text{退款} = \text{否} | \text{否}) = 4/7$
- $P(\text{退款} = \text{是} | \text{是}) = 0$
- $P(\text{退款} = \text{否} | \text{是}) = 1$
- $P(\text{婚姻状况} = \text{单身} | \text{否}) = 2/7$

- $P(\text{婚姻状况} = \text{离异} | \text{否}) = 1/7$
- $P(\text{婚姻状况} = \text{已婚} | \text{否}) = 4/7$
- $P(\text{婚姻状况} = \text{单身} | \text{是}) = 2/7$
- $P(\text{婚姻状况} = \text{离异} | \text{是}) = 1/7$
- $P(\text{婚姻状况} = \text{已婚} | \text{是}) = 0$

对于应纳税收入：

- 若类别 = 否：样本均值 = 110，样本方差 = 2975
- 若类别 = 是：样本均值 = 90，样本方差 = 25

朴素贝叶斯：优缺点

对孤立噪声点具有鲁棒性（Robust）。

在概率估计计算过程中，可通过忽略包含缺失值的实例来处理缺失值问题。

对不相关属性具有鲁棒性。

独立性假设对于某些属性可能不成立：

- 相关属性的存在会降低朴素贝叶斯分类器的性能。

相关属性示例

- 两个属性 A 、 B 以及类别 Y （均为二元变量）
- 先验概率：
 - $P(Y = 0) = P(Y = 1) = 0.5$
- 属性 A 的类条件概率：
 - $P(A = 0 | Y = 0) = 0.4$ ， $P(A = 1 | Y = 0) = 0.6$
 - $P(A = 0 | Y = 1) = 0.6$ ， $P(A = 1 | Y = 1) = 0.4$
- 属性 B 的类条件概率与属性 A 相同
- 当 $Y = 0$ 时， B 与 A 完全相关；当 $Y = 1$ 时， B 与 A 相互独立
- 需要对一条 $A = 0$ ， $B = 0$ 的记录进行分类
- $P(Y = 0 | A = 0, B = 0)$

$$\begin{aligned}
 &= \frac{P(A = 0, B = 0 | Y = 0)P(Y = 0)}{P(A = 0, B = 0)} \\
 &= \frac{P(A = 0 | Y = 0)P(B = 0 | Y = 0)P(Y = 0)}{P(A = 0, B = 0)} \\
 &= (0.16 * 0.5) / P(A = 0, B = 0)
 \end{aligned}$$

- $P(Y = 1 | A = 0, B = 0)$

$$\begin{aligned}
&= \frac{P(A = 0, B = 0|Y = 1)P(Y = 1)}{P(A = 0, B = 0)} \\
&= \frac{P(A = 0|Y = 1)P(B = 0|Y = 1)P(Y = 1)}{P(A = 0, B = 0)} \\
&= (0.36 * 0.5) / P(A = 0, B = 0)
\end{aligned}$$

- 因此预测结果为 $Y = 1$

其他贝叶斯分类器

- 当怀疑属性之间可能存在相关性时：
- 可以使用诸如贝叶斯信念网络（BBN）之类的其他技术。
- 贝叶斯信念网络使用图形模型（网络）来捕捉特定领域的先验知识，以及变量之间的因果依赖关系。

■ [Week 8]

决策树和KNN

[Decision-Tree Classifier Tutorial](#)

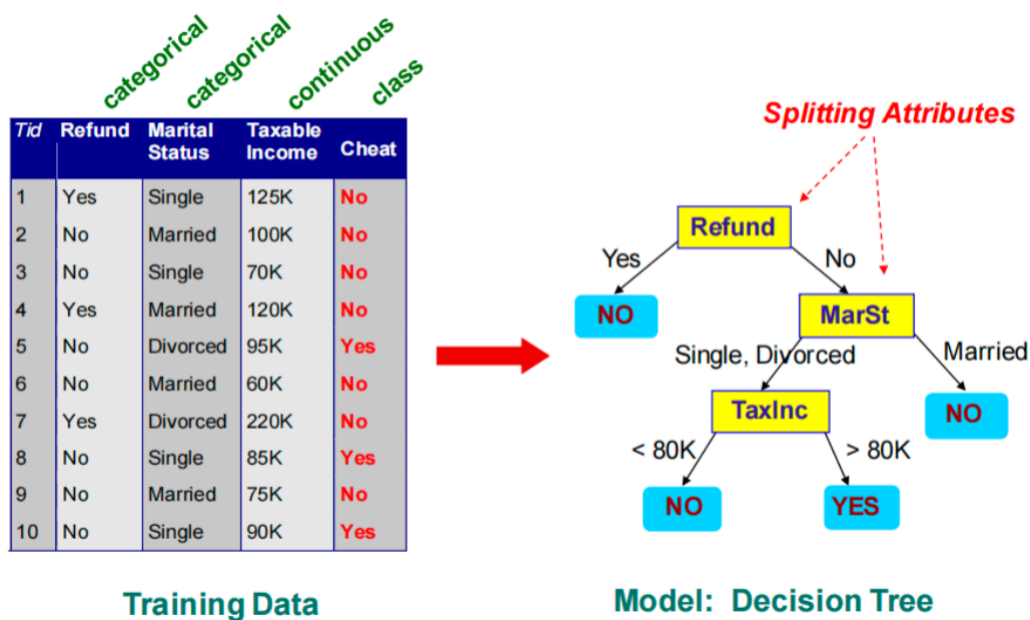
[kNN Classifier Tutorial](#)

[1] 决策树

决策树背后的逻辑

针对给定记录提出一系列问题

- 每个问题都围绕其中一个属性展开
- 对一个问题的回答决定了接下来要问的问题（或者是否还需要问下一个问题）
- 持续提问，直到能够推断出给定记录所属的类别



决策树的结构

决策树：一种层次结构

- 一个根节点：无入边，有零条或多条出边
- 内部节点：恰有一条入边，有两条或更多出边
- 叶节点或终端节点：恰有一条入边，无出边

每个叶节点都被分配一个类别标签

每个非叶节点都包含针对某一属性的测试条件

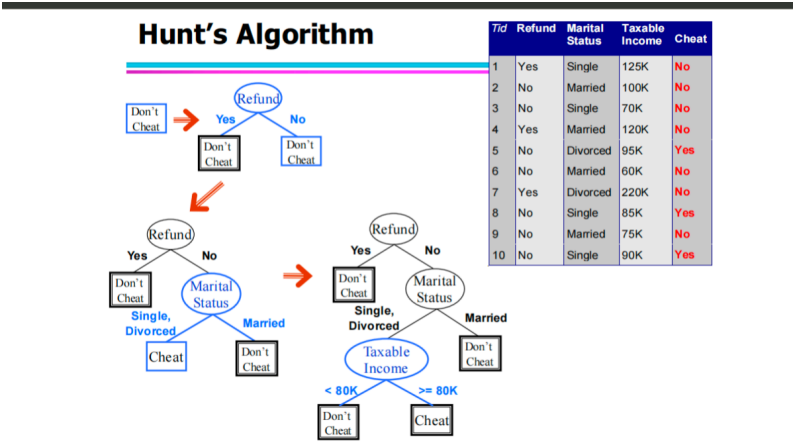
决策树学习中的挑战

- 从给定的属性集可以构建出数量呈指数级增长的决策树
 - 其中一些决策树比其他的更“准确”，或者说是更好的分类器
 - 找到最优决策树在计算上是不可行的
- 有高效算法可在合理时间内学习到相当准确（尽管可能是次优的）的决策树
 - 采用贪心策略
 - 对接下来用于划分数据的属性做出局部最优选择

决策树归纳

- 多种算法：
 - 亨特算法（最早的算法之一）
 - 分类与回归树（CART）
 - ID3算法、C4.5算法
 - 可扩展的线性时间归纳算法（SLIQ）、可扩展的并行规则归纳算法（SPRINT）

亨特算法的一般结构



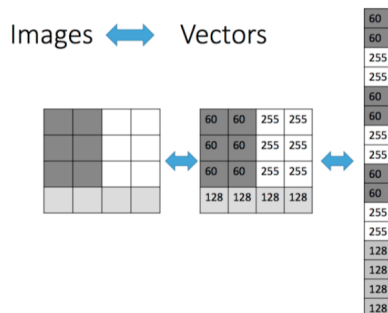
- 设 D_t 为到达节点 t 的训练记录集
- 一般步骤：
 - 若 D_t 中的记录都属于同一类别 y_t ，则节点 t 为叶节点，标记为 y_t 。
 - 若 D_t 为空集，则节点 t 为叶节点，标记为默认类别 y_d 。
 - 若 D_t 中的记录属于多个类别，使用属性测试将数据划分为更小的子集。对每个子集递归应用该步骤。

输入向量

- 机器学习算法需要处理多种类型的数据：图像、文本、音频波形、信用卡交易记录等。
- 常用策略：将输入表示为 $\mathbb{R}^d$ 空间中的输入向量
 - 表示：映射到另一个易于处理的空间
 - 向量是一种很好的表示方式，因为我们可以运用线性代数！
- 可以使用原始像素：

- 图像 $\rightleftharpoons$ 向量
- （图示：图像矩阵转换为数值矩阵，再转换为向量的过程）
- 如果能计算出有意义的特征向量，效果会更好。

Can use raw pixels:



Can do much better if you compute a vector of meaningful features.

- 从数学角度看，我们的训练集由一系列输入向量 $\mathbf{x} \in \mathbb{R}^d$ 及其对应的目标值（即标签） t 组成
 - **回归**： t 是一个实数（如股票价格）
 - **分类**： t 是离散集合 $\{1, \dots, C\}$ 中的一个元素
 - 如今， t 通常是高度结构化的对象（如图像）
- 将训练集记为 $\{(\mathbf{x}^{(1)}, t^{(1)}), \dots, (\mathbf{x}^{(N)}, t^{(N)})\}$
 - 注意：这里的上标与指数运算无关！

[2] 最近邻算法

- 假设我们有一个新的输入向量 $\mathbf{x}$ 需要分类。
- 算法思路：在训练集中找到与 $\mathbf{x}$ 最接近的输入向量，并将其标签作为 $\mathbf{x}$ 的标签。
- 可以用欧几里得距离来定义“最接近”：

$$\|\mathbf{x}^{(a)} - \mathbf{x}^{(b)}\|_2 = \sqrt{\sum_{j=1}^d (x_j^{(a)} - x_j^{(b)})^2}$$

算法步骤

1. 从存储的训练集中找到与 $\mathbf{x}$ 最接近的样本 $(\mathbf{x}^*, t^*)$ ，即：

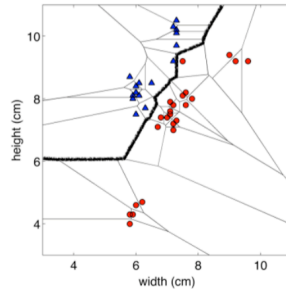
$$\mathbf{x}^* = \underset{\mathbf{x}^{(i)} \in \text{train. set}}{\operatorname{argmin}} \operatorname{distance}(\mathbf{x}^{(i)}, \mathbf{x})$$

2. 输出 $y = t^*$ 。

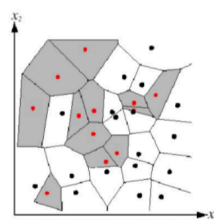
- 注意：我们不需要计算平方根。为什么呢？

决策边界

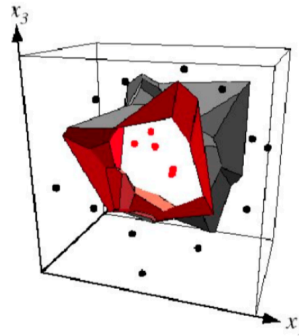
决策边界：输入空间中被划分到不同类别的区域之间的边界。



我们可以使用沃罗诺伊图（Voronoi diagram）来可视化分类场景中的算法表现。



3D 决策边界



最近邻算法对噪声或错误标记的数据（“类别噪声”）敏感。有解决办法吗？

通过让k个最近邻进行投票来平滑分类结果。

算法

1. 找到与测试实例 $\mathbf{x}$ 最接近的k个样本 $\{\mathbf{x}^{(i)}, t^{(i)}\}$ 。

2. 分类输出为多数类：

$$y = \arg \max_{t^{(z)}} \sum_{r=1}^k \delta(t^{(z)}, t^{(r)})$$

1 近邻 (1 NN) :

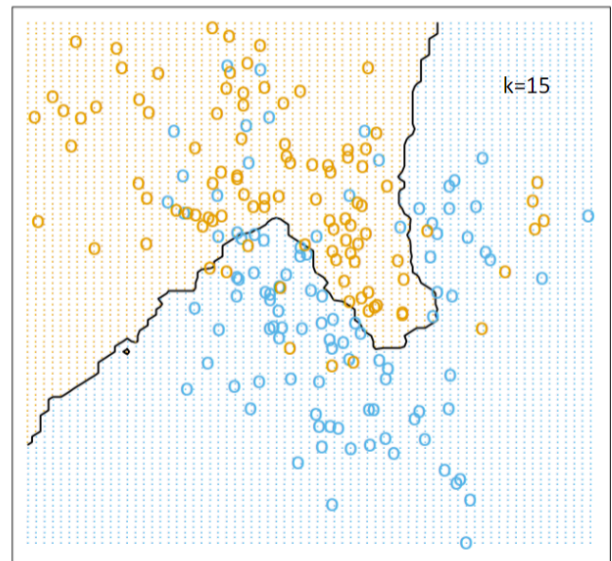
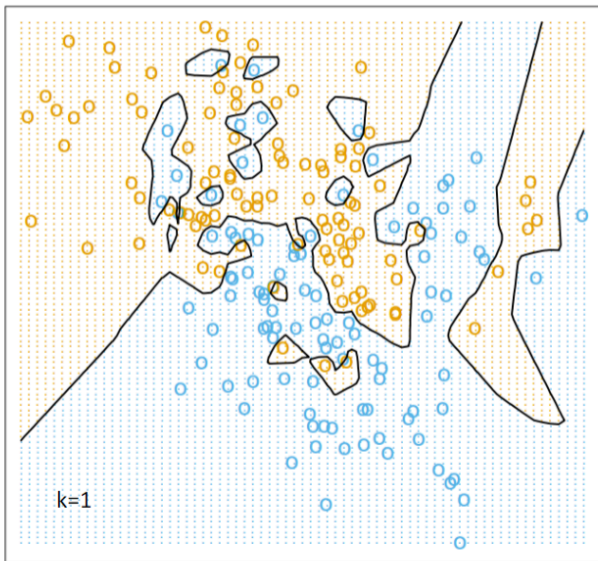
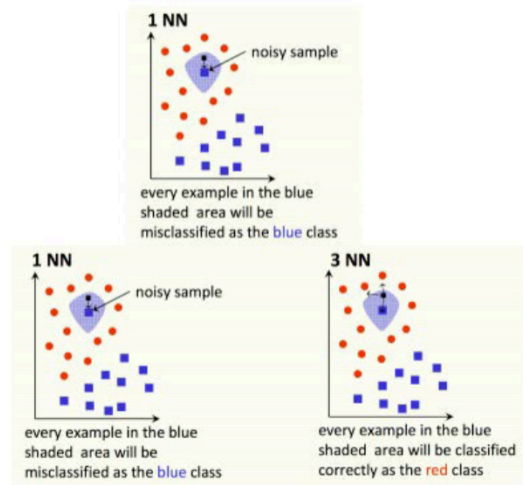
- 含噪样本 (noisy sample)。蓝色阴影区域内的每个样本都会被误分类为蓝色类别。

1 近邻 (1 NN) :

- 含噪样本 (noisy sample)。蓝色阴影区域内的每个样本都会被误分类为蓝色类别。

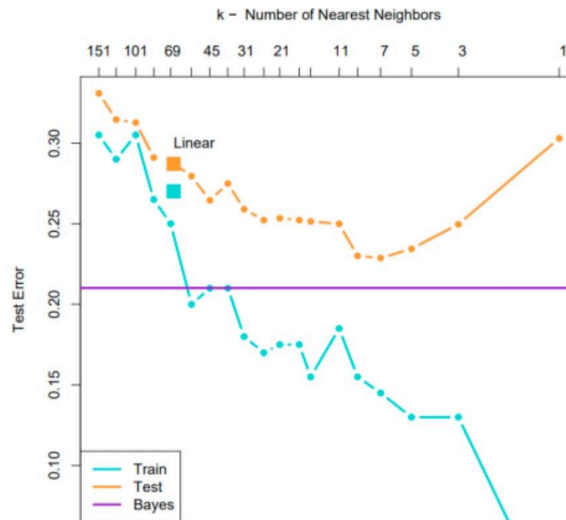
3 近邻 (3 NN) :

- 蓝色阴影区域内的每个样本都会被正确分类为红色类别。



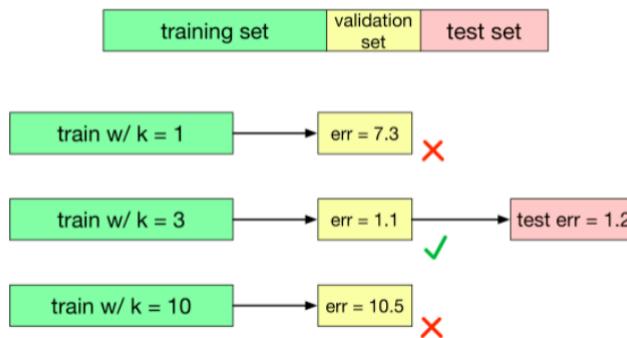
选择k值时的权衡

- **k值较小**
 - 善于捕捉细粒度模式
 - 可能过拟合，即对训练数据中的随机特性敏感
- **k值较大**
 - 通过对大量样本求平均做出稳定预测
 - 可能欠拟合，即无法捕捉重要规律
- 经验法则： $k < \sqrt{n}$ ，其中 n 是训练样本数量
- 我们希望算法能够对其之前未见过的数据具有泛化能力。
- 我们可以使用测试集来衡量泛化误差（在新样本上的错误率）。



验证集与测试集

- k 是超参数的一个例子，它无法作为学习算法本身的一部分进行拟合。
- 我们可以使用验证集来调整超参数：

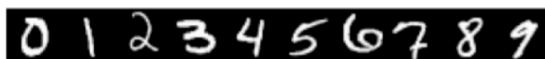


- 训练集、验证集、测试集划分示意（分别以绿色、黄色、粉色表示）
- 使用 $k = 1$ 训练，误差 $err = 7.3$ （验证结果不佳，标记红色叉）
- 使用 $k = 3$ 训练，误差 $err = 1.1$ ，测试误差 $test\ err = 1.2$ （验证结果良好，标记绿色对勾）
- 使用 $k = 10$ 训练，误差 $err = 10.5$ （验证结果不佳，标记红色叉）
- 测试集仅在最后用于衡量最终配置的泛化性能。

示例：数字分类

示例：数字分类

- 数据量充足时表现良好



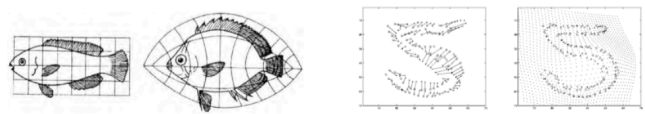
扬·勒昆 (Yann LeCun) 的 MNIST 数字识别

- 手写数字
- 28×28 像素图像，维度 $d = 784$
- 60,000 个训练样本
- 10,000 个测试样本

	Test Error Rate (%)	
Linear classifier (1-layer NN)	12.0	线性分类器（单层神经网络）
K-nearest-neighbors, Euclidean	5.0	k近邻（欧氏距离）
K-nearest-neighbors, Euclidean, deskewed	2.4	k近邻（欧氏距离，去倾斜）
K-NN, Tangent Distance, 16x16	1.1	k近邻（切线距离，16×16）
K-NN, shape context matching	0.67	k近邻（形状上下文匹配）
1000 RBF + linear classifier	3.6	1000个径向基函数 + 线性分类器
SVM deg 4 polynomial	1.1	支持向量机（4阶多项式）
2-layer NN, 300 hidden units	4.7	两层神经网络，300个隐藏单元
2-layer NN, 300 HU, [deskewing]	1.6	两层神经网络，300个隐藏单元（去倾斜）
LeNet-5, [distortions]	0.8	LeNet 5（含扭曲）
Boosted LeNet-4, [distortions]	0.7	增强型LeNet-4（含扭曲）

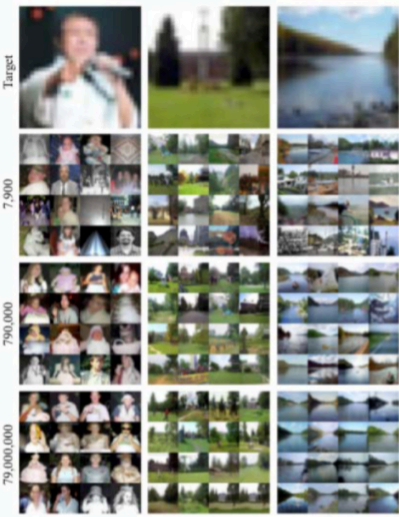
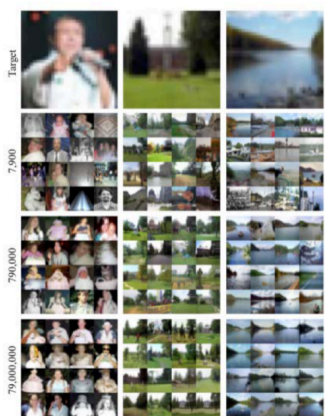
###

- 若采用良好的相似度量，k近邻算法（KNN）能有更优表现。
- 例如：用于目标识别的形状上下文。为实现对图像变换的不变性，尝试对一幅图像进行变形以匹配另一幅图像。
 - 距离度量：变形后图像上对应点之间的平均距离
- 在MNIST数据集上实现了0.63%的错误率，相比基于欧氏距离的KNN的3%错误率有显著提升。
- 当时可与卷积网络相媲美，但需要精细的工程设计。



示例：八千万张小图像

- “八千万张小图像” 是首个超大型图像数据集，由缩至 32×32 尺寸的彩色图像组成。
- 借助大型数据集，能找到更优的语义匹配，k 近邻算法（KNN）也能有出色表现。
- 注意：这需要精心选择相似度量。



示例：八千万张小图像

- “八千万张小图像” 是首个超大型图像数据集，由缩至 32×32 尺寸的彩色图像组成。
- 借助大型数据集，能找到更优的语义匹配，k 近邻算法（KNN）也能有出色表现。
- 注意：这需要精心选择相似度量。

优缺点

- 算法简单，所有工作都在测试阶段完成——从某种意义上说，无需学习过程！
- 可通过改变 k 值来控制模型复杂度。
- 受维度灾难影响。

■ [Week 9]

[Interview](#)

[使用机器学习进行 IRIS 分类：基础知识](#)

[1] 应用机器学习时

假设你训练了一个图像分类器，其准确率为80%。接下来该怎么做？

- 增加更多数据？
- 延长算法训练时间？
- 使用更大的模型？
- 添加正则化？
- 添加新特征？

我们将探讨如何**对决策进行优先级排序（Prioritize decision）**，以构建高性能的机器学习系统。

开发机器学习模型时，通常会使用三个数据集：

- **训练集 Training Set**：用于训练算法的数据。
- **开发集 Development Set**（验证集或留出集）：用于调优算法的数据。
- **测试集 Test Set**：用于评估模型最终性能的数据。

这些数据集的典型使用方式如下：

1. **训练**：尝试一个新模型，并在训练集上进行拟合。
2. **模型选择**：使用评估指标在开发集上评估性能。根据结果，回到步骤1尝试新的模型思路。
3. **评估**：最后，在测试集上评估模型在实际场景中的性能。

开发集与测试集

- **开发集 Dev Set**：其目的是检测模型性能的变化。
- **测试集**：其目的是可靠地评估模型在实际场景中的性能。
- 在开发集上选择最佳模型会高估其性能。因此，一个未经处理的纯净测试集至关重要！

选择开发集和测试集

如何选择开发集和测试集？我们强调两个需要考虑的因素。

分布一致性

开发集和测试集应来自我们在实际应用中会遇到的数据分布。

- 这是因为我们希望优化模型在部署时的性能（例如，狗图像的分类准确率）。
- 训练数据可能来自不同的分布（例如，包含其他类型的动物图像）。

数据集规模

开发集和测试集需能够评估模型未来的性能。

- 传统建议是预留30%的数据用于开发集和测试集。
- 在大数据场景下，30%可能过多。例如，从100万个数据中选取3000个用于测试就足够了。

模型选择

这里，我们再次强调两个需要考虑的因素。

选择评估指标

模型开发流程需要优化单一性能指标。

- 如果多个指标都很重要，可以将它们合并为一个（例如，精确率 + 召回率 $\rightarrow$ F值）。
- 其他指标（例如，计算性能）可以作为约束条件。

更新模型

我们根据开发集性能选择超参数，方法包括：

- 凭直觉，例如，如果模型过拟合，增加正则化。
- 超参数网格搜索（hyperparameter grid search）或随机搜索。
- 有时，我们会使用专门的算法，如贝叶斯超参数优化（Bayesian hyperparameter optimization）。

机器学习工作流程的局限性

你可能会遇到一些问题：

1. 在开发集上反复选择最佳模型后，出现开发集过拟合。
2. 开发集和测试集可能不再代表真实的数据分布。
3. 评估指标可能不再能衡量真实性能。

在这种情况下，你需要收集更多数据，并且/或者更改评估指标。

[2] 评估分类模型

分类准确率

对于分类算法，最简单且最直观的评估指标是准确率：

$$\text{acc}(f) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}\{f(x) = y\}$$

其中 $\mathbb{I}\{\cdot\}$ 是指示函数（输入为真时取值为1，否则为0）。

混淆矩阵

通过混淆矩阵，我们能更好地理解分类错误。

- 纵轴表示真实类别 y_j 。
- 横轴表示预测类别 $\hat{y}_k$ 。
- 单元格 $(y_j, \hat{y}_k)$ 中的数值表示真实类别为 y_j 但被预测为 $\hat{y}_k$ 的样本数量。

二分类评估指标

在进行二分类任务时，我们可以用更精确的方式评估模型性能。考虑以下混淆矩阵：

	预测为正类 $\hat{y} = 1$	预测为负类 $\hat{y} = 0$
正类 $y = 1$	真正例 (TP)	假负例 (FN)
负类 $y = 0$	假正例 (FP)	真负例 (TN)

我们可以将准确率定义如下：

$$\text{准确率} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{FN} + \text{TN}}$$

它是正确预测的数量除以预测的总数量。

敏感度与特异度

	预测为正类 $\hat{y} = 1$	预测为负类 $\hat{y} = 0$
正类 $y = 1$	真正例 (TP)	假负例 (FN)
负类 $y = 0$	假正例 (FP)	真负例 (TN)

我们也可以分别查看每个类别的“准确率”，这有助于发现类别不平衡问题。

敏感度（召回率，真正例率）

$= \frac{\text{TP}}{\text{正类样本数}} = \frac{\text{TP}}{\text{TP} + \text{FN}}$

特异度（真负例率）

$= \frac{\text{TN}}{\text{负类样本数}} = \frac{\text{TN}}{\text{TN} + \text{FP}}$

精确率与召回率

	预测为正类 $\hat{y} = 1$	预测为负类 $\hat{y} = 0$
正类 $y = 1$	真正例 (TP)	假负例 (FN)
负类 $y = 0$	假正例 (FP)	真负例 (TN)

另一组评估指标是精确率和召回率：

精确率（阳性预测值）

$= \frac{\text{TP}}{\text{预测为正类的样本数}} = \frac{\text{TP}}{\text{TP} + \text{FP}}$

召回率（敏感度，真正例率）

$= \frac{\text{TP}}{\text{正类样本数}} = \frac{\text{TP}}{\text{TP} + \text{FN}}$

F值

F值是精确率和召回率的几何平均值。

$$F值 = \frac{2}{\frac{1}{精确率} + \frac{1}{召回率}}$$

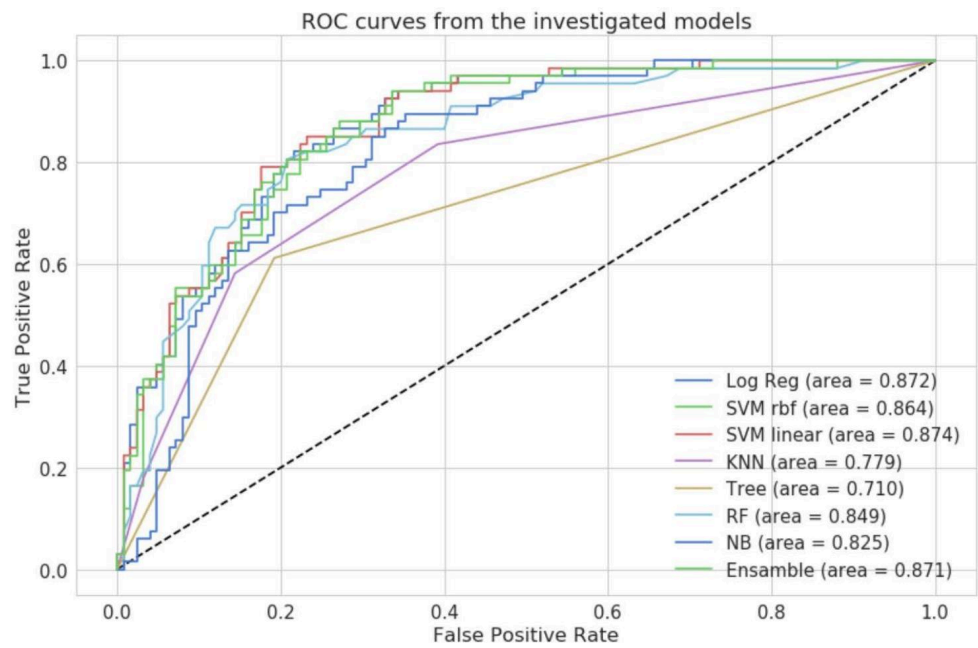
在精确率和召回率都达到完美时，F值为1；若精确率或召回率其中之一为0，F值则为0。

受试者工作特征曲线（ROC）

在二分类任务中，受试者工作特征曲线（ROC）会在改变正例判定阈值时，绘制真正例率（TPR）和假正例率（FPR）。

$$TPR（召回率，敏感度） = \frac{TP}{正类样本数} = \frac{TP}{TP + FN}$$
$$FPR（1 - 特异度，真负例率） = 1 - \frac{TN}{负类样本数} = \frac{FP}{TP + FP}$$

模型与 ROC 曲线等的比较



曲线下面积

我们可以使用曲线下面积（AUC）作为评估分类器性能的单一指标。

- 理想分类器（ROC曲线到达左上角）的ROC - AUC值为1。
- 随机分类器（对角线虚线）的ROC - AUC值为0.5。

[3] 误差分析

理解模型性能的一个关键环节是系统地检查其误差。误差分析就是这样一个可以实现该目标的规范流程。

确定模型改进的优先级：示例

假设你训练了一个用于动物照片分类的图像分类器。

- 你听说有些狗被误分类成了猫。
- 你可以选择集成一个商用狗分类器，但这需要一个月时间来实施。
- 你是否应该推进这个集成工作？

做出这一决策的关键在于仔细检查模型的实际性能。

假设你需要决定是否值得修复某类错误。

- 首先，从开发集中随机收集约100个被误分类的样本。
- 查看这些样本，并统计其中出现该错误的数量。
如果在误分类的样本中，只有5%存在该问题，那么这可能并不重要。但如果50%存在该问题，那么这就很重要了。

误差分析系统地识别模型最常出现的错误。具体步骤如下：

1. 从开发集中被误分类的样本里随机抽取一个子集。
2. 人工检查这些样本，确定最常见的错误类型或类别。
3. 统计每个错误类别所影响的样本占比。

你应该优先处理最常见的错误类别。

假设你刚训练了一个新的图像分类器。

- 查看前20 - 30个错误后，你发现这些错误都是由于图像模糊、翻转或标注错误导致的。
- 这些就构成了错误类别。

然后我们浏览随机抽取的错误子集，并将它们归入相应类别。

我们知道，最重要的修复工作是纠正翻转的图像。

	模糊	翻转	标注错误
图像1	X	X	
图像2		X	
图像3			X
...			
总计	20%	50%	30%

开发集规模

开发集规模应该多大？误差分析给出了一个下限。

- 你应该至少有50 - 100个标注错误的样本用于分析。
- 如果模型的错误率为 ϵ ，我们期望开发集规模约为 $100/\epsilon$ 。
 - 例如，若错误率为5%，我们期望开发集规模约为2000。

此外，记得定期更新开发集，以尽量减少过拟合。

训练集上的误差分析

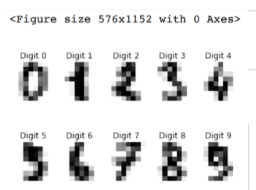
我们应该在开发集还是训练集上进行误差分析呢？

- 归根结底，我们关注的是在实际应用中会出现的错误。在开发集上进行分析能最好地评估这些错误。
- 然而，如果模型在训练集上就出现错误，那么在开发集上也可能会出现同样的错误。

因此，分析和修正训练集上的错误同样重要。

误差分析：另一个示例

我们来看一个在小型玩具数据集上进行误差分析的例子。



我们将使用来自scikit - learn的数字数据集，它是MNIST数据集的一个缩简版本。

```
1 In [2]:  
2 from sklearn.datasets import load_digits  
3 digits = load_digits()
```

我们可以按如下方式可视化这些数字：

```
1 In [3]:  
2 from matplotlib import pyplot as plt  
3  
4 plt.figure(figsize=(8, 16))  
5 _, axes = plt.subplots(2, 5)  
6 images_and_labels = list(zip(digits.images, digits.target))  
7 for ax, (image, label) in zip(axes.flatten(), images_and_labels[:10]):  
8     ax.set_axis_off()  
9     ax.imshow(image, cmap=plt.cm.gray_r, interpolation='nearest')  
10    ax.set_title('Digit %i' % label)
```

我们将这些数据分成规模相等的训练集和开发集。

```

1 In [4]:
2 # https://scikit-
  learn.org/stable/auto_examples/classification/plot_digits_classification.html
3 from sklearn.model_selection import train_test_split
4
5 # 将数据拆分为训练集和测试集子集
6 n_samples = len(digits.images)
7 data = digits.images.reshape((n_samples, -1))
8 X_train, X_dev, y_train, y_dev = train_test_split(
9     data, digits.target, test_size=0.5, shuffle=False)

```

我们可以在这些数据上训练一个简单的Softmax模型。

```

1 In [5]:
2 # https://scikit-
  learn.org/stable/auto_examples/classification/plot_digits_classification.html
3 from sklearn.linear_model import LogisticRegression
4 classifier = LogisticRegression(max_iter=1e7)
5
6 # 我们在一半的数据上训练数字分类器
7 classifier.fit(X_train, y_train)
8
9 # 现在对另一半数据的数字值进行预测
10 predicted = classifier.predict(X_dev)

```

它达到了如下准确率。

```

1 In [6]:
2 (predicted == y_dev).mean()
3 Out[6]: 0.9332591768631813

```

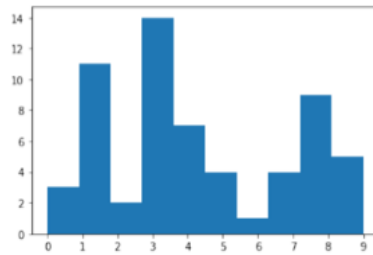
我们假设某些数字比其他数字更容易被误分类。

```

1 In [7]:
2 # 这些开发集中的数字被错误分类
3 X_error = X_dev[predicted != y_dev]
4 y_error = y_dev[predicted != y_dev]
5 p_error = predicted[predicted != y_dev]
6
7 # 这些开发集中的数字被正确分类
8 X_corr = X_dev[predicted == y_dev]
9 y_corr = y_dev[predicted == y_dev]
10 p_corr = predicted[predicted == y_dev]
11
12 # 显示直方图
13 plt.xticks(range(10))
14 plt.hist(y_error)

```

```
Out[7]: (array([ 3., 11.,  2., 14.,  7.,  4.,  1.,  4.,  9.,  5.]),  
         array([0. , 0.9, 1.8, 2.7, 3.6, 4.5, 5.4, 6.3, 7.2, 8.1, 9. ]),  
         <BarContainer object of 10 artists>)
```



最常被误分类的数字是3。我们可以通过查看被误分类的3的子集（第一行）并与被正确分类的3（第二行）进行比较，来研究这个问题。

```
1 In [13]:  
2 plt.figure(figsize=(8, 16))  
3 _, axes = plt.subplots(2, 8)  
4  
5 # 这些图像被错误分类  
6 images_and_labels = list(zip(X_error[y_error==3], p_error[y_error==3]))  
7 for ax, (image, label) in zip(axes[0, :], images_and_labels[:8]):  
8     ax.set_axis_off()  
9     ax.imshow(image.reshape((8, 8)), cmap=plt.cm.gray_r, interpolation='nearest')  
10    ax.set_title('f(x)=%i' % label)  
11  
12 # 这些图像被正确分类  
13 images_and_labels = list(zip(X_corr[y_corr==3], p_corr[y_corr==3]))  
14 for ax, (image, label) in zip(axes[1, :], images_and_labels[:8]):  
15     ax.set_axis_off()  
16     ax.imshow(image.reshape((8, 8)), cmap=plt.cm.gray_r, interpolation='nearest')  
17     ax.set_title('f(x)=%i' % label)
```

我们发现模型对特定样式的3会误分类，我们可以将精力集中在这类错误上。

<Figure size 576x1152 with 0 Axes>

f(x)=9 f(x)=5 f(x)=8 f(x)=7 f(x)=7 f(x)=8 f(x)=8 f(x)=5
3 3 3 3 3 3 3 3

f(x)=3 f(x)=3 f(x)=3 f(x)=3 f(x)=3 f(x)=3 f(x)=3 f(x)=3
3 3 3 3 3 3 3 3

误差分析的局限性

误差分析的主要局限性包括：

1. 特别容易对开发集过拟合，因为我们优先修复开发集上的错误。
2. 误差分析可能很耗费精力（但仍然重要！）。
3. 某些较大的趋势（如对数据过拟合）可能不太容易通过误差分析发现。

因此，我们需要进行其他分析来解释和诊断错误。

偏差/方差分析

理解模型性能的另一方法是考察它对数据的过拟合或欠拟合程度。我们将此称为偏差/方差分析。

过拟合(Overfitting)是机器学习中最常见的失效模式之一。

- 一个表达能力很强的模型（如高次多项式）能完美拟合训练数据集。
- 但该模型在训练集之外会做出非常错误的预测，不具备泛化能力。

容易过拟合的算法也被称为具有高方差，因为如果我们对数据集稍作扰动，它输出的预测模型就会有很大变化。

欠拟合(Underfitting)是机器学习中另一个常见问题。

- 模型过于简单，无法很好地拟合数据（例如，用线性回归去近似高次多项式）。
- 因此，该模型在训练数据上不准确，在新数据上也不准确。

由于模型无法拟合数据，我们称其具有高偏差。

论偏差与方差的重要性

机器学习中的每一个错误，都源于欠拟合（偏差）或者过拟合（方差）。

- **偏差**：模型无法从数据中提取有效信息。
- **方差**：所提取的信息无法推广到新数据上。

从定义来看，如果一个模型没有偏差和方差，那它就是一个完美模型。因此，理解偏差和方差非常重要。

量化偏差与方差

我们可以按如下方式近似量化模型的偏差和方差：

$$\text{开发集误差} = \underbrace{(\text{开发集误差} - \text{训练集误差})}_{\text{方差}} + \underbrace{\text{训练集误差}}_{\text{偏差}}$$

- **偏差**：简单来说就是训练集误差（模型能否拟合训练数据？）
- **方差**：是开发集上超出训练集误差的部分（模型能否泛化到开发集上？）

我们可以依据这些结论在实际中诊断偏差/方差问题。

- 如果训练集误差高，说明存在高偏差。
- 如果开发集误差远高于训练集误差，说明存在高方差。

看下面的例子：

- 训练集误差为15%。
- 开发集误差为16%。
- 人类水平误差为2%。

这是高偏差（欠拟合）的典型例子。

再看另一个例子：

- 训练集误差为4%。
- 开发集误差为12%。
- 人类水平误差为2%。

这是高方差（过拟合）的例子。

解决方差问题

降低方差的最佳方法是为模型提供更多数据。

- 过拟合的出现是因为我们没有足够的数据来拟合一个表达能力强的模型。
- 因此，增加数据后，模型死记硬背的解决方案就不再适用。

然而，由于计算成本或数据获取成本过高，这可能并不可行。降低方差的替代方法包括：

- **添加正则化**：L2 正则化、随机失活（Dropout）、提前终止训练（会增加偏差）。
- **舍弃或改变特征**：理想情况下，舍弃无关特征（需要人工分析）。
- **减小模型规模**：减小层的规模、内核复杂度等。正则化通常效果更好。

在降低偏差和方差时，我们可以利用误差分析来指导改进方向，例如：

- 确定要添加或移除哪些特征。
- 确定要收集哪些额外数据。

这两种分析揭示不同类型的问题：

- 误差分析找出系统性错误（例如，由于缺少狗的训练图像，导致在狗的图像上频繁出错）。
- 偏差/方差分析找出一般性错误（例如，模型死记硬背训练集，缺乏泛化能力）。

这两种分析（误差分析与偏差/方差分析）也相互补充。

- 偏差/方差分析指出哪些错误类别以及哪些数据集（训练集、开发集）对误差分析有用。
- 误差分析确定修复高偏差或高方差的具体方法（例如，添加哪些新特征）。

偏差/方差分析与超参数搜索

偏差/方差分析也有助于指导超参数搜索：

- 超参数通常通过网格搜索或随机搜索来选择。
- 偏差/方差分析为搜索方向提供思路。
- 例如，如果出现过拟合，我们就需要搜索更大的正则化参数值。

模型迭代循环

总之，机器学习模型开发可视为在以下两个步骤间交替进行：

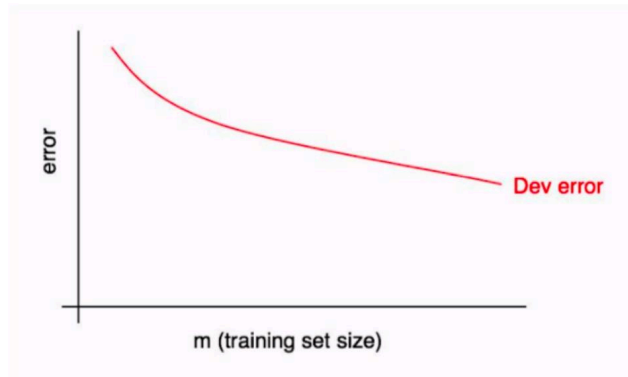
- **减少欠拟合**（如增大模型规模）
- **减少过拟合**（如提供更多数据）

误差分析在这个过程中指导具体的改进方向。

[4] 诊断模型性能的工具

学习曲线

学习曲线展示模型性能随训练集规模变化的情况。



学习曲线是在超参数固定的情况下定义的。可以看到，随着提供给模型的数据增多，开发集误差会降低。

损失曲线

许多算法使用梯度下降等迭代优化过程来最小化损失函数。

损失曲线绘制训练目标（值）随训练步数的变化情况，可基于训练集或开发集绘制。

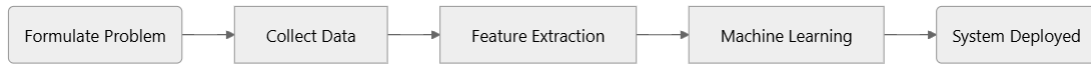
验证曲线

机器学习模型通常有超参数，比如L2正则化强度、神经网络层的规模、K - 均值聚类的簇数等。

验证曲线绘制模型性能随超参数取值变化的情况，可基于训练集或开发集绘制。

■ [Week 10]

[1] 人工智能系统



1. 问题构建→数据收集→特征提取→机器学习→系统部署
2. 计算智能：特征→标签（统一）
 - 分类
 - 异常检测
3. 感知智能：特征→标签（非统一）
 - 情感检测
4. 创造智能
 - 超越当前生成模型
5. 通用智能

[2] 人工智能应用

- 多媒体信息检索
 - 计算机视觉
 - 计算机听觉系统（音乐/语音/一般音频）
 - 自然语言处理
 - 多模态媒体（视频）
- 数据挖掘
- 模式识别

[3] 计算机视觉

- 关键任务
 - 目标检测/识别
 - 图像分割
 - 场景理解（问答系统）
 - 运动分析
- 应用领域
 - 自动驾驶汽车
 - 医学成像
 - 增强现实
 - 监控系统

[4] 计算机视觉技术

- **传统方法**
 - 边缘检测（坎尼、索贝尔算法）
 - 哈尔级联
 - 卷积
- **深度学习**
 - 卷积神经网络（CNNs）
 - 视觉变换器（ViT）
 - YOLO（实时目标检测）
- **工具**
 - OpenCV
 - PyTorch/TensorFlow
 - MMDetection

[5] 计算机听觉系统 - 关键任务

- **语音处理**
 - 自动语音识别（ASR）
 - 说话人识别/验证
 - 语音情感识别
- **音乐分析**
 - 音乐流派/艺术家分类
 - 节拍/节奏检测
 - 音乐生成/混音
- **一般音频处理**
 - 环境声音检测（如玻璃破碎声、警报声）
 - 音频事件定位
 - 噪声消除

[6] 音频处理技术

- **传统方法**
 - 用于语音特征提取的梅尔频率倒谱系数（MFCCs）
 - 用于音频序列建模的隐马尔可夫模型（HMMs）
 - 用于声源分离的非负矩阵分解（NMF）
- **深度学习**
 - 用于处理时间相关性的循环神经网络/长短期记忆网络（RNN/LSTM，如语音转文本）
 - 用于多语言自动语音识别的Transformer模型（如Whisper）

- 用于高保真音乐合成的扩散模型
- 工具
 - Librosa (音频特征提取)
 - Kaldi (语音识别工具包)
 - PyTorch Audio / TensorFlow Signal
 - Audacity + 人工智能插件 (如降噪)

[7] 神经语言处理 (NLP)

关键任务

- 句法层面
 - 分词/词性标注
 - 依存句法分析
- 语义层面
 - 命名实体识别 (NER)
 - 情感分析
 - 机器翻译
- 生成任务
 - 文本摘要
 - 对话系统 (聊天机器人)
 - 创意写作 (诗歌/故事)

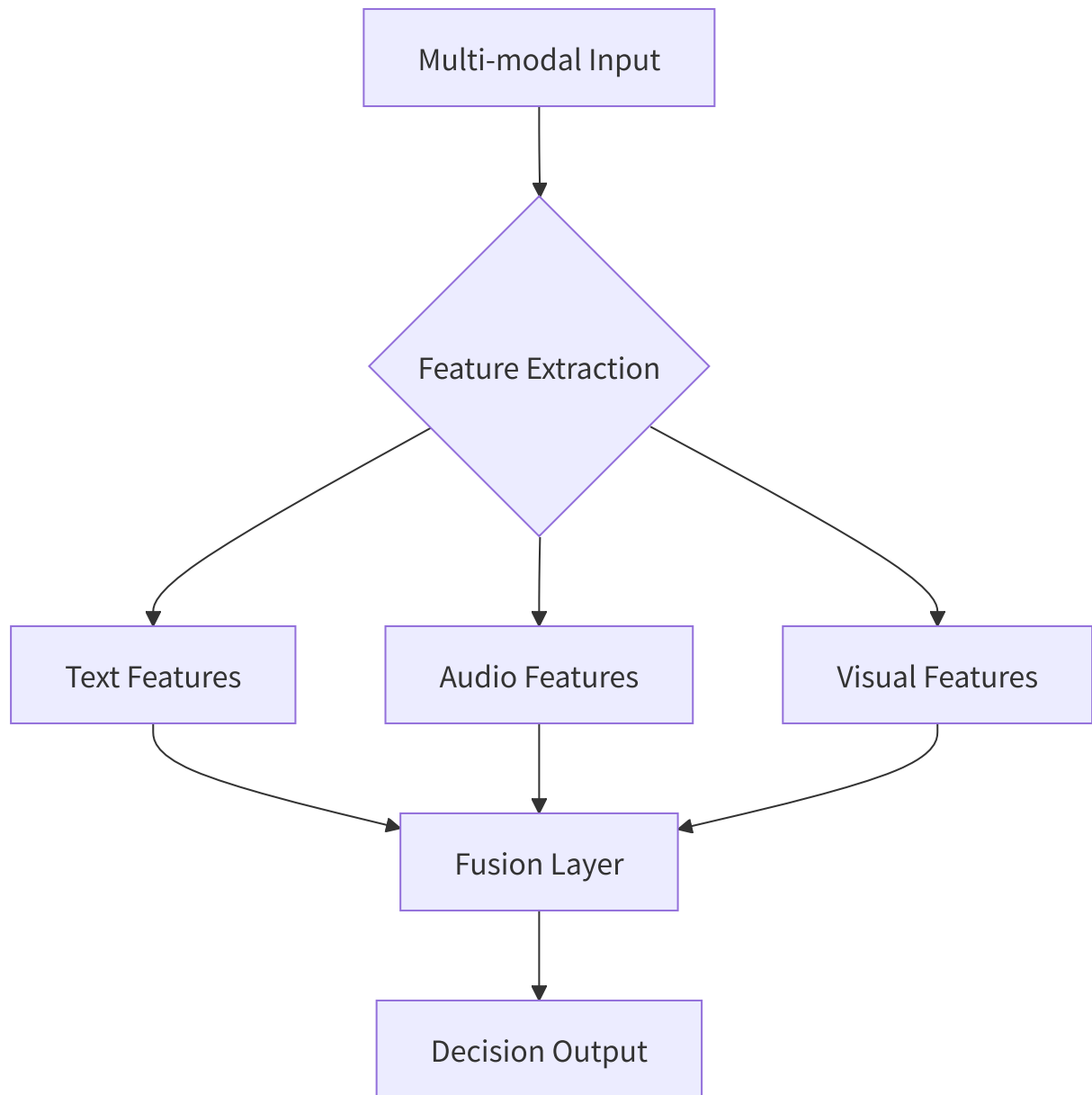
[8] 自然语言处理技术

- 传统方法
 - 词袋模型 (BoW)
 - 词频 - 逆文档频率加权 (TF-IDF)
 - 隐马尔可夫模型 (HMMs)
- 深度学习
 - 循环神经网络/长短期记忆网络 (RNNs/LSTMs)
 - Transformer模型 (BERT、GPT)
 - 注意力机制
- 工具
 - NLTK/Spacy (语言处理工具)
 - HuggingFace Transformers
 - 斯坦福CoreNLP

[9] 多模态媒体处理

核心挑战

- 跨模态对齐（如音视频同步）
- 特征融合策略
- 上下文感知理解



[10] 多媒体人工智能技术的应用

- 视频理解
 - 动作识别（体育分析）
 - 基于内容的视频检索
- 人机交互
 - 手势和语音控制系统
 - 多模态情感识别
- 教育领域
 - 结合幻灯片分析的自动讲座转录

- 多感官学习助手

[11] 其他人工智能应用 I

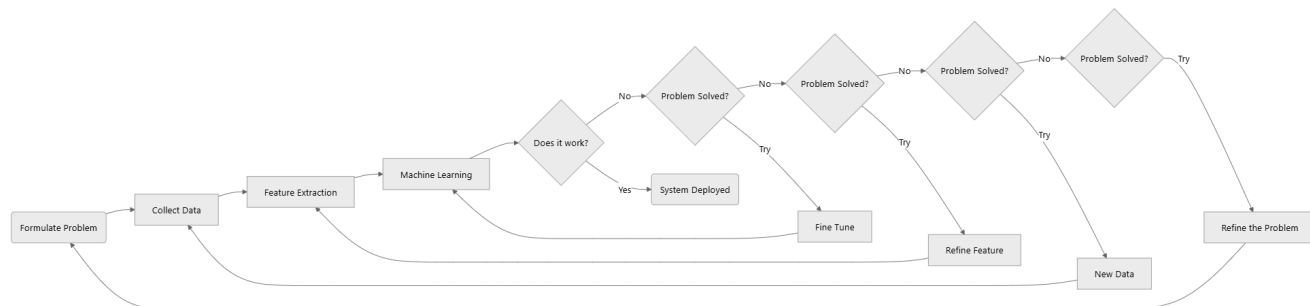
- 气候与环境科学
 - 气候建模：人工智能利用大量数据集预测天气模式、极端事件或长期气候趋势。
 - 野生动物保护：机器学习算法分析相机陷阱或卫星图像，以追踪濒危物种并打击偷猎行为。
- 农业领域
 - 精准农业：人工智能利用卫星图像和物联网传感器优化灌溉、施肥和害虫防治。
 - 作物产量预测：模型根据天气、土壤状况和历史数据预测收成。

[12] 其他人工智能应用 II

- 金融与经济
 - 算法交易：人工智能通过分析市场趋势和新闻舆情来执行高频交易。
 - 信用评分：机器学习利用非传统数据（如消费习惯、社交媒体活动）评估信用价值。
- 网络安全
 - 威胁检测：人工智能识别网络流量中的异常情况以防止入侵（如Darktrace的自学习系统）。
 - 网络钓鱼防范：自然语言处理模型分析电子邮件中的恶意意图或仿冒企图。
- 制造与供应链
 - 预测性维护：人工智能通过分析传感器数据预测设备故障，减少停机时间（如西门子的人工智能驱动工厂）。
 - 物流优化：算法简化配送路线和库存管理（如联合包裹服务公司的ORION系统）。

■ [Week 11]

[1] AI 系统



[2] 相关问题

存在的问题

- 数据隐私
- 数据收集（偏差）
 - 数据可得性
- 算法偏差
- 评估偏差
- 系统安全
- 虚假媒体
- 系统功耗

应对措施

- 匿名化（是否真正有效？）
- 了解数据群体（尤其是背景信息）
- 了解算法
- 了解偏好
- 提高系统稳健性
- 深度伪造检测
- 算法简化

[3] 匿名化学习

也叫**联邦学习 Federated Learning**

- 理想情况下，数据可以集中管理。
- 然而，数据传输存在数据泄露风险。
- 数据存储在本地上机器上，因此可能存在以下问题：

- 特征定义不一致（或数据收集不一致）
- 数据质量不一致
- 特征分布不一致

[4] 数据收集偏差

思考一下：机器学习的主要目的是什么？

- 本质上，机器学习试图在两个特征空间（如标签空间和数据空间）之间建立映射。
- 标签空间和数据空间需要用概率分布来描述。

接着，思考一下训练数据集是如何起作用的？

- 理想情况下，训练数据集能够估计真实概率分布。
- 但训练数据集的概率分布与真实情况（数据偏差）永远不会相同。

[5] 数据收集偏差

- 如何构建一个优质的数据集
 - 涵盖足够的多样性
 - 拥有足够的数据量
- 如果无法涵盖所有类型的数据会怎样？
 - 不同的情况意味着不同的领域
 - 存在领域偏移的机器学习（一种迁移学习）
 - 领域泛化
 - 领域自适应
- 如果没有足够的数据会怎样？
 - 少样本学习/单样本学习/零样本学习

[6] 算法偏差

- 当使用机器学习算法时，会引入一些假设：
 - 高斯混合模型（GMM）：所有聚类都可以用高斯分布近似
 - 支持向量机（SVM）：存在一个特征空间，可以用线性边界对数据进行分类
 - 朴素贝叶斯：各因素的影响相互独立
- 因此，需要使引入的假设能很好地适配所处理的问题

[7] 系统安全

人工智能系统可能受到攻击，以下行为会误导人工智能系统：

- **扰动**：添加微小的不可见数据噪声
- **数据投毒**：向数据集中添加少量数据

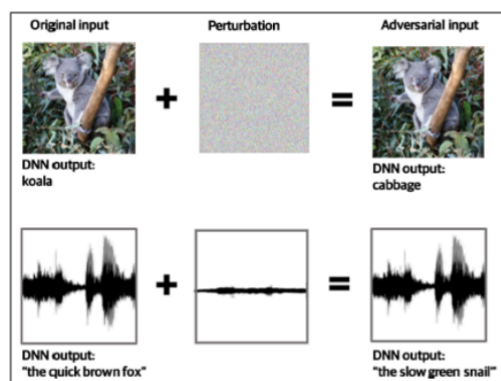


Figure 1-3. Adversarial perturbation applied across an image to fool an image classifier and across audio to fool a speech-to-text system

[8] 深度伪造

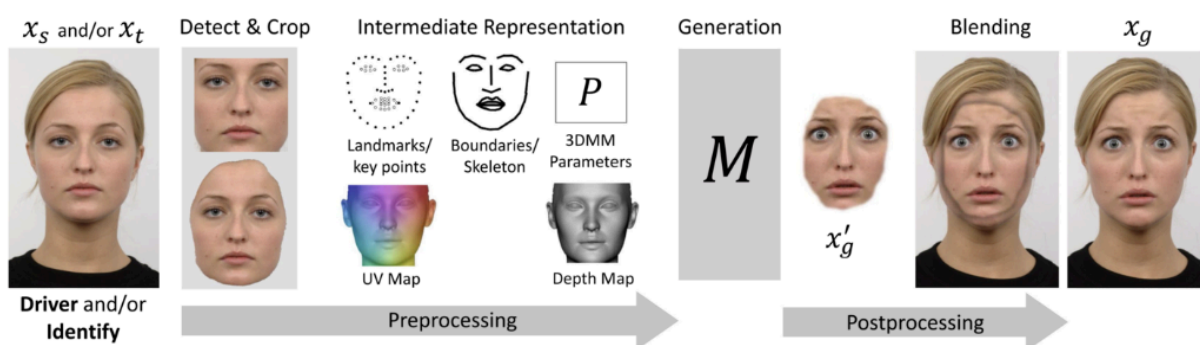


图5. 制作重演和换脸深度伪造的处理流程。通常仅执行这些步骤的一个子集。

[9] 功耗（环保角度）

问题：阿尔法狗（AlphaGO）击败了柯洁，这是真的吗？

- 柯洁比赛消耗的能量：500千卡 $\approx$ 0.0005千瓦时
- 阿尔法狗比赛消耗的能量：60千瓦时

因此，有必要尽可能简化机器学习算法（尤其是深度学习系统）。

[10] 未来挑战

存在的问题

- 不同情境下特征分布不同
- 数据不平衡
- 数据可得性低
- 验证域中出现新类别
- 在新环境中自动适应

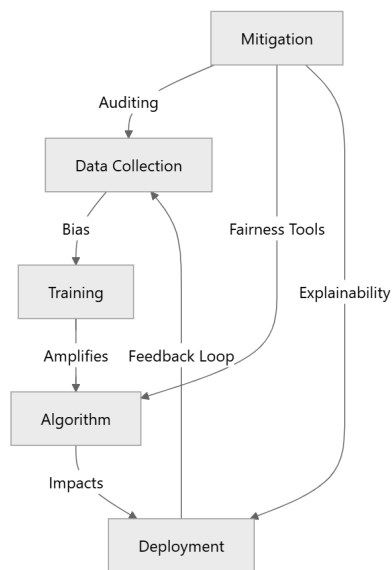
应对技术

- 领域泛化/领域自适应、迁移学习
- 少样本学习/异常检测
- 少样本学习/单样本学习

- 开放集学习/零样本学习
- 增量学习

[11] 未来发展方向

- **可解释人工智能 (XAI)**：通过可解释性来审视偏差来源（如局部可解释模型无关解释（LIME）、SHAP值）。
- **监管框架**：如欧盟《人工智能法案》、算法问责法案。
- **设计即伦理**：在人工智能从问题构建到部署的整个生命周期的每个阶段都嵌入偏差检查。



问题不在于机器能否思考，而在于人类能否设计出符合伦理道德的会思考的机器。